

From SFC circulars to reviewable and verifiable rules

LegalMath: a regulatory specification workbench for Hong Kong private banking

A proposal for compliance, operations, technology and model-risk teams

The proposed product turns a regulatory change into a source-linked specification, worked cases, executable decisions and evidence of what has been checked. Legal interpretation remains visible throughout. A bounded prototype will test whether this makes changes easier to review and less likely to introduce material errors.

The proposed investment

Private-bank regulatory change repeatedly crosses a difficult boundary: a circular describes what the institution should do, while software requires exact inputs, conditions, dates and consequences. Much of the interpretation occurs in documents, meetings and tickets that are only loosely connected to the eventual code. A small change to an exception, client category or monitoring event can therefore survive ordinary testing while changing the treatment of clients.

LegalMath would make that boundary a product. Its central deliverable is a *reviewable regulatory specification*: the relevant source passages, their interpretation, typed rules, unresolved questions, examples, and the evidence supporting execution. Compliance officers review the meaning in ordinary language and worked cases; engineers implement or integrate the same versioned rules; tests and formal checks report precisely which properties hold. A flowchart is one view of this specification, alongside decision tables and event histories.

One complete example makes that proposition testable. The joint SFC–HKMA SPI circular permits specified streamlining for certain clients and transactions. A client at the wealth threshold can still fail because of conservative objectives, an unselected product category, missing agreement or withdrawn consent. The worked case follows a synthetic client through the circular and both annexes, the interpretation and evidence record, a typed specification, distinguishing tests, generated Java and a separate Java host call. It retains offering documents, requested explanations and relevant warnings instead of turning streamlining into a general permission to trade.

This proposal recommends a staged prototype centred on the SFC–HKMA sophisticated professional investor (SPI) circular and its annexes, with separate amendment and marketing cases. Start with reviewed rules, a small interpreter, deterministic explanations and independent test scenarios. Add language-model drafting and counterexample search once that baseline works. The bank’s required output is Java. A deterministic compiler produces source and a JAR from the adopted specification; Python supports authoring and reference checks. The delivered narrow demonstration implements exact money, explicit unknown/conflict results, source-linked rule traces and consent replay. The larger workbench must add review, the full language and event contracts, data normalization and controlled release. Formal languages are explained from their basic ingredients before the literature survey: expressions and types for decisions, exceptions for qualified rules, and states and events for agreements. Existing MathDevMCP, DynareMCP and ResearchAssistant components can support proof evidence, investigative search and source management through narrow adapters.

The initial commercial decision is whether to fund a limited discovery and prototype exercise. The exercise should measure review effort, materially wrong conclusions, missed obligations and amendment handling against the existing manual process. The proposed schedule is twelve weeks with named compliance participation. Timing and staffing are planning assumptions; no savings or accuracy result has yet been measured. The study includes 38 downloaded paper editions representing 37 works, all seven requested sources, foundational normative logic, and standards and software documentation. The full implementation contract supplies 35 decision cases plus negative, event and release scenarios. The separate Java demonstration executes 32 SPI decision cases and eleven consent histories, compares the complete results with a reference implementation, and detects three deliberately introduced rule defects. These checks establish a working path to Java for the stated subset. They do not establish autonomous legal translation, complete regulatory coverage or bank production readiness. The prototype plan names the remaining modules, interfaces and acceptance conditions, including a mandatory Java backend.

Contents

The proposed investment	1
1 The problem: a regulatory decision loses meaning as it becomes software	4
1.1 The regulatory perimeter is part of the specification	4
1.2 A running case: sophisticated professional investors	5
1.3 Amendment, actor and conditional language: tokenised products	7
1.4 Marketing rules require exceptions and judgment	8
1.5 What makes the current work expensive and hard to verify	8
2 What a formal legal language actually does	8
2.1 From a sentence to a calculation with a defined meaning	9
2.2 Catala: general rules and explicit exceptions	9
2.3 Stipula: agreements, authorized actions and state changes	10
2.4 What it means to verify such a model	12
2.5 Solvers, explanations and search in ordinary programming terms	13
3 A complete worked change: circular 23EC35 to a Java decision	13
3.1 The business question and the facts the bank would need	13
3.2 Collect the whole instruction before extracting rules	14
3.3 Record the interpretation and who supplies each fact	16
3.4 Construct and evaluate the executable specification	18
3.5 Generate, build and call the Java release candidate	19
3.6 Withdraw consent, replay the decision, then change the rule	20
4 What the literature contributes	21
4.1 Scope and method of this review	21
4.2 From statutory text to a language of definitions and exceptions	21
4.3 Why exceptions, duties and permissions need different logics	22
4.4 Dates and test generation expose defects that ordinary examples miss	23
4.5 Legal rules need authority, unknown facts and duties	24
4.6 Stipula and the transition from a decision to a contract history	26
4.7 Consistency of clauses and compliance of processes	27
4.8 Language models and the formalization boundary	28
4.9 Software choices and the role of standards	31
4.10 How to turn the citation network into a development program	33
5 The product: a shared specification that can be reviewed and executed	33
5.1 What a reviewer should see	33
5.2 The intermediate representation	34
5.3 A precise example: money, alternatives and unknown values	37
5.4 Obligations and event histories	37
5.5 What can be proved, and what must be reviewed	39
5.6 Counterexamples make disagreements reviewable	40
5.7 The smallest useful execution architecture	41
5.8 Language-model drafting with visible uncertainty	44
5.9 Amendments and the release package	44
6 Delivering a control to the bank's Java system	45
6.1 What has actually been built for this proposal	45
6.2 The input and output boundary	45
6.3 How the compiler emits final Java rather than a suggestion	47
6.4 Reproduce the build and see the host result	47
6.5 The bank adapter and the exposure race	48
6.6 Release contents, extension tasks and acceptance	49
7 A prototype that can answer a product decision	50

7.1	Scope, team and deliverables	50
7.2	A startable implementation sequence	50
7.3	Reusing ResearchAssistant, MathDevMCP and DynareMCP	52
7.4	Evaluation must measure both meaning and implementation	53
7.5	Targeted tests and counterexamples	55
7.6	Economic value and operating cost	55
7.7	Risks that shape the implementation	56
8	The first acceptance cases and the research record	56
8.1	Conformance cases and the research record	57
8.2	Decisions needed at the start of a bank prototype	58

1 The problem: a regulatory decision loses meaning as it becomes software

A private bank must turn regulatory requirements into client classifications, product permissions, disclosures, monitoring, records and staff actions. Those requirements rarely arrive as a complete algorithm. A circular may change one part of an existing regime, depend on definitions in a code, distinguish product providers from distributors, or explain an exception in an annex. Software must nevertheless decide what to do for a particular client, product and event.

The sponsor's reported problem is the error-prone manual reading and updating of bank systems. This proposal treats that account as the starting business need; it does not assume a measured incident rate. The process below is a plausible operating model to validate with the bank. A compliance officer reads a circular, writes an interpretation, and discusses it with product and operations staff. A business analyst turns the interpretation into requirements. Engineers implement configuration or code, and testers construct examples from those requirements. Each participant can perform competently while a condition disappears between documents. When the circular changes, the bank must reconstruct the chain.

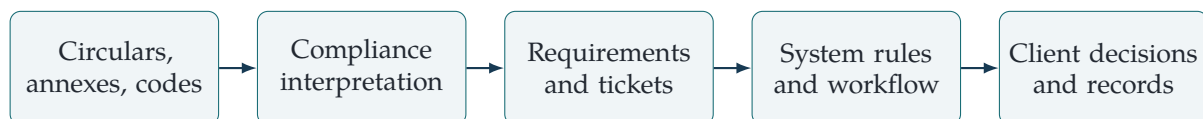


Figure 1: The proposed discovery exercise will map the bank's actual process onto these transitions. LegalMath preserves the source, interpretation and tests across them, so a reviewer can follow a decision back to its reason.

The product opportunity is to make that chain inspectable and reusable. A reviewer should be able to select a rule and see the passage that supports it, the interpretation adopted, the facts it needs, the situations in which it changes an outcome, and the version actually used. A subsequent amendment should identify the affected controls and test cases without relying on the memory of the original implementation team.

1.1 The regulatory perimeter is part of the specification

The supplied Hong Kong Securities and Futures Commission (SFC) product-authorization page is a useful discovery source, but it is not a complete inventory of a private bank's distribution obligations. The SPI guidance examined here is a joint circular from the SFC and Hong Kong Monetary Authority (HKMA) to intermediaries. The tokenisation circular distinguishes product-provider responsibilities from distributor responsibilities and imports other product and intermediary rules. The marketing circular refers to the Securities and Futures Ordinance, the Code of Conduct and advertising guidance. These dependencies determine the meaning of the extracted clauses (SFC, 2026a; SFC and HKMA, 2023a; SFC, 2026b, 2023a).

Discovery therefore starts with the bank's legal entities and activities. For a specified business, the inventory should record whether the institution acts as distributor, product provider, adviser or another regulated actor, and which client and product classes are involved. Being a Hong Kong office of a US banking group does not by itself settle any of these questions. The prototype will record the selected Hong Kong perimeter and leave other group and jurisdictional requirements as separately identified dependencies. A broader rollout requires the bank's authoritative inventory.

Document classification also matters. The September 2025 joint circular announces a concur-

rent thematic review of distribution practices and describes what the review will examine. It should enter regulatory-change tracking; its description of the review is not, by itself, a new numerical client-eligibility test (SFC and HKMA, 2025). A translator that treats every sentence as a binding conditional will manufacture rules. Conversely, a translator that extracts only sentences containing a particular modal verb can miss definitions, incorporated requirements and material explanatory guidance.

LegalMath should preserve a passage's role: definition, requirement, permission, exception, supervisory expectation, explanation or announcement. The original wording and its authority remain alongside the adopted implementation. A bank may choose an additional operational restriction, but that choice should be recorded as bank policy with an owner and rationale. It must not be silently attributed to the SFC.

1.2 A running case: sophisticated professional investors

The joint circular of 28 July 2023 supplies an unusually useful test case because its apparent simplicity conceals several different kinds of rule. Annex 1 defines the qualifying criteria, eligible investment transactions, information and consent requirements, and continuing controls. Annex 2 clarifies difficult implementation cases. The examples below are synthetic and illustrate the cited provisions; they do not complete an SPI assessment or authorize a transaction (SFC and HKMA, 2023a,b,c).

A one-word error changes the financial condition

Annex 1, paragraph 3.1 provides two alternative financial routes: a portfolio of at least HK\$40 million, or net assets excluding the primary residence of at least HK\$80 million, including the stated foreign-currency equivalents. Consider a client with an eligible portfolio valued at HK\$35 million and qualifying net assets of HK\$90 million. The second route meets this financial condition. Replacing the alternative with a conjunction rejects the client incorrectly. Replacing either inclusive comparison with a strict comparison rejects a client exactly at that threshold. Both defects are easy to miss when test data use wealth amounts far from the boundary (SFC and HKMA, 2023b, Annex 1, para. 3.1).

The numbers also require definitions. The value of assets held at the bank is not automatically the qualifying portfolio, and total wealth is not automatically net assets excluding the primary residence. Paragraph 3.3 specifies the net-asset calculation. In a simple synthetic case with HK\$120 million of assets, including a HK\$30 million primary residence, and HK\$15 million of liabilities, that calculation gives HK\$75 million: $120 - 15 - 30$. Omitting the residence gives HK\$105 million and changes the result of the net-asset route. The portfolio route must still be assessed independently. The bank must also establish how the source definitions map to its valuation and ownership records (SFC and HKMA, 2023b, Annex 1, paras. 3.2–3.4).

Ownership and missing evidence can reverse the result

For a joint portfolio with persons other than the client's associate, paragraph 3.2(c) refers to the client's share specified in a written agreement, or an equal share in the absence of that agreement. Suppose a two-holder portfolio is worth HK\$60 million and the written allocation gives the client 20 percent. The relevant share for this illustration is HK\$12 million, not the whole account and not an automatically assumed half. The rule for a joint account with an associate is separately stated. A generic "joint-account multiplier" loses this distinction (SFC and HKMA, 2023b, Annex 1, para. 3.2).

An unavailable document is also different from an established absence of an agreement. The bank must decide what evidence establishes each fact. If a feed returns a null portfolio value, replacing null with zero creates a substantive conclusion from a data failure. If qualifying net assets are known to be HK\$90 million, the financial condition can still be established by that route. If they are HK\$70 million, the unknown portfolio value can change the answer; the financial condition remains unresolved. The product must express that distinction directly.

Wealth does not establish suitability for streamlined treatment

Paragraph 1 combines professional-investor status, the financial criteria, knowledge or experience, and investment objectives at the relevant date. Paragraph 4.1 requires reasonable satisfaction concerning sophistication and sets out alternative supporting criteria. Paragraph 5.1 excludes conservative clients from SPI treatment for this guidance. A wealthy client whose objective is capital preservation therefore demonstrates why a field named `wealth_qualified` must not automatically populate `streamlined_approach_available` (SFC and HKMA, 2023b, Annex 1, paras. 1, 4–5).

Some criteria are countable; others require assessment. A degree, a qualification, relevant work experience and prior transactions are distinct routes. A translator should not invent a score that trades off wealth against sophistication or investment objectives. The assessment and its supporting evidence need an owner, date and review record. Paragraph 2 also supplies a separate route for a qualifying investment-holding corporation wholly owned by SPIs. It cannot be represented by treating every client identifier as an individual.

Experience and consent belong to a product category

Suppose the client relies on the transaction-experience route and has completed five bond transactions in the relevant three-year period. That history does not automatically establish the same route for an unrelated product category. Paragraph 7.3 connects category eligibility to the alternative qualification and experience routes; paragraph 7.2 requires the client's choice of categories to be documented and category information to be provided. A global client flag can therefore remain true while a proposed category is outside the selected scope (SFC and HKMA, 2023b, Annex 1, paras. 4.1, 7.1–7.3).

The identity of the person matters too. Annex 2, FAQ 2 explains the assessment where a client acts through a power of attorney. The attorney's sophistication cannot simply replace the client's qualifying attributes. In software terms, the customer, account holder, representative, beneficial owner and person giving an instruction are different roles. Collapsing them into one record can make every subsequent logical step internally consistent and still apply it to the wrong person (SFC and HKMA, 2023c, Annex 2, FAQ 2).

Exposure monitoring is an event rule

Paragraph 8.3 permits two monitoring approaches. One concerns gross exposure on execution of transactions under the streamlined approach. The other uses designated accounts and checks the specified condition after a top-up or deposit of new funds. Annex 2, FAQ 6 explains how these approaches operate, including transferred exposures and restrictions on additional funds when a designated account exceeds the threshold. FAQ 7 addresses leverage (SFC and HKMA, 2023b,c, Annex 1, para. 8.3; Annex 2, FAQs 6–7).

A single rule that rejects every transaction whenever an account's current value exceeds a thresh-

old loses the distinction between these approaches. In the designated-account example, the FAQ allows continued execution above the threshold subject to its conditions restricting new funds. The monitoring mode, movement of funds, gross exposure and prior events all matter. An appreciated position, a leveraged purchase, a cash withdrawal and a transferred non-cash position must not be reduced to the same change in account market value. This is a strong reason to model event histories as well as decision tables.

Consent and continuing obligations outlive the initial decision

Paragraph 13 calls for prior written agreement and an explanation of the ability to withdraw from, or change, the arrangement. Paragraph 14 requires annual review and reminders. Assessment and agreement are therefore not permanent facts. A transaction considered before receipt of a withdrawal instruction and one considered afterwards require different state snapshots. A review reminder being queued is different from being sent; being sent is different from the review being completed (SFC and HKMA, 2023b, Annex 1, paras. 12–14).

The streamlined approach also retains specific information and conduct requirements. The solicited or recommended transaction provisions and the unsolicited complex-product provisions have different contexts. A broad “skip suitability and disclosure” switch would conceal these remaining duties (SFC and HKMA, 2023b, Annex 1, paras. 9–11). The prototype should return the relevant decision and the associated tasks, explanations and evidence requests, rather than a single apparently comprehensive compliance flag.

1.3 Amendment, actor and conditional language: tokenised products

The SFC’s 20 April 2026 tokenisation circular replaces the November 2023 circular; the earlier public entry is marked accordingly. A source collector that keeps only a URL and the most recently extracted text cannot reliably reconstruct which requirements an older decision used. A collector that retains both but fails to record replacement can apply inconsistent generations together (SFC, 2023b, 2026b).

The 2026 circular also provides useful clause-level challenges. Paragraph 10 keeps responsibility for the tokenisation arrangement and ownership records with product providers despite outsourcing. Paragraph 18 addresses distributors, including providers distributing their own products. These are actor-specific requirements: the word “bank” is not a sufficient applicability condition. Paragraph 13 conditions the use of public-permissionless networks on additional proper controls; translating it into an unconditional technology ban changes the statement (SFC, 2026b, paras. 10, 13, 18).

Paragraphs 14–16 illustrate several different triggers. Confirmation and demonstration upon request are distinguishable actions. Third-party verification and a legal opinion are conditioned on an SFC request in the cited paragraphs. Removing that trigger creates a different obligation; removing the obligation creates the opposite error. A request must be represented as an event, with its subject and responsible actor, and any deadline must come from a relevant source or separately identified operational policy. The product must not invent a generic deadline (SFC, 2026b, paras. 14–16).

Prior consultation and prior approval are also different. Paragraphs 20–21 and footnote 8 distinguish new products, tokenisation of existing products, material changes, and circumstances concerning tokenised share classes. Turning “consultation required” into “approval obtained” skips an action and invents a result. The proposed workbench will expose such verbs and their dependencies as separate obligations and evidence fields (SFC, 2026b, paras. 20–21 and n. 8).

1.4 Marketing rules require exceptions and judgment

The October 2023 distributor circular examines additional returns and incentives, redemption restrictions and marketing of fund-related services. Its treatment of guaranteed-return arrangements depends on the arrangement and the relevant legal classification. A potential structured-product classification should produce a supported assessment task; a language model should not resolve it from a slogan alone (SFC, 2023a, paras. 5–11).

The circular's discussion of gifts expressly distinguishes discounts of fees or charges. Losing that exception makes an apparently conservative implementation incorrectly broad. Its redemption discussion distinguishes administrative procedures or cut-off times from reducing a fund's dealing frequency. A daily dealing fund offered with weekly redemption presents a different issue from a different daily processing cut-off. These are excellent paired test cases because they differ in the fact that determines the treatment (SFC, 2023a, paras. 9–14).

The advertising discussion addresses misleading impressions and comparisons with deposits. Keyword detection can nominate material for review, but it cannot establish the whole classification: meaning depends on the presentation and the service described. The system should retain the advertisement, the relevant source passages and the reviewer's conclusion. That is a useful result even when there is no defensible fully automated predicate (SFC, 2023a, paras. 15–17).

1.5 What makes the current work expensive and hard to verify

The common failure is loss of context. Thresholds detach from definitions; exceptions detach from their parent rules; duties detach from actors and events; and historical decisions detach from the source version that justified them. Document search helps locate text, but it does not preserve those relationships through implementation. More fluent summaries can make an omitted condition harder to notice.

Review then becomes repetitive. Compliance officers must infer what an engineer understood, engineers must infer what a compliance memo leaves implicit, and testers must construct enough cases to expose mismatches. If tests are written only from the implementation ticket, a mistranslation shared by the ticket and code can pass every test. After an amendment, teams may retest too little because dependencies are hidden, or retest too much because they cannot show what is unaffected.

The proposed product makes each material interpretation a maintained part of the system. Its value should be measured in fewer material defects, more complete explanations, and less repeated review effort. Those benefits are hypotheses for the prototype to test. The evidence needed to test them includes independently reviewed cases, disagreements that remain visible, and the time spent resolving them; a high extraction score alone cannot establish the business case.

2 What a formal legal language actually does

Consider the instruction to obtain written agreement before applying the streamlined approach. A checklist can remind an officer to obtain it. A database can store a scanned agreement. Neither tells a program what to conclude if the agreement covers bonds while the proposed transaction is in a different category, or if the client withdrew yesterday. A formal language supplies a precise way to state the relevant conditions and the changes that events produce. Its purpose here is to preserve an agreed interpretation closely enough that people can inspect it and computers can execute and check it.

The word “formal” does not mean that the language resolves legal judgment. It means that a well-formed statement has a specified computational meaning. We can then distinguish disagreement about that meaning from a programming mistake in carrying it out. The original circular, the adopted interpretation and the formal statement must remain connected.

2.1 From a sentence to a calculation with a defined meaning

A programming language needs three ingredients. Its *syntax* says which statements are allowed. Its *types* say which kinds of value can be combined. Its *semantics* says what each allowed statement does. For example, `portfolio >= HKD(40000000)` has comparison syntax, compares two money values, and means an inclusive monetary comparison. Comparing that portfolio with a consent date is a type error. Using `>` instead of `>=` is well-typed but changes the meaning at the boundary. A type checker catches the first error; an independently chosen boundary example catches the second.

For the SPI financial condition, a readable formal statement is:

Listing 1: Illustrative controlled syntax for Annex 1 paragraph 3.1. This is a teaching notation, not Catala or Stipula source.

```
financial_condition(client, date) :=
  portfolio(client, date) >= HKD(40000000)
  OR net_assets_ex_home(client, date) >= HKD(800000000)
```

Here `client` and `date` are arguments: every occurrence refers to the same person and valuation date. `portfolio` and `net_assets_ex_home` name defined inputs. The source, paragraphs 3.2–3.4, determines the ownership and net-asset definitions; the host bank must provide values calculated under the adopted mapping. The notation makes no assumption that the portfolio is simply assets held at this bank.

The compiler stores the statement as a tree. The root is `OR`; its two children are inclusive comparisons; each comparison has an input and a monetary constant. This tree is called an *abstract syntax tree*: it retains the operations and their arguments without depending on the layout of the original sentence. The JSON in Section 5 is a serialized version of such a tree. An intermediate representation, or IR, adds stable identifiers, source locations, types and interpretation records to that executable structure.

Evaluation works from the leaves towards the root. For a portfolio of HK\$35 million and net assets of HK\$90 million, the comparisons give false and true; their alternative gives true. Missing portfolio evidence produces an unresolved comparison. An established net-asset route can still make the alternative true. If neither route is established and one remains unresolved, the answer remains unresolved. This is why an ordinary Java `boolean`, which has only two values, is insufficient as the complete result type.

The financial predicate answers one question. It does not produce a duty or an authorization to execute an order. The larger model must combine qualification, the selected product category, consent, exposure and retained requirements. Keeping these named subquestions separate is useful both to the reviewer and to the engineer diagnosing a rejected request.

2.2 Catala: general rules and explicit exceptions

Catala was designed around a recurring structure in legislation: a value is defined by a general rule, subject to exceptions. Its central contribution is to give that structure an explicit executable

meaning, while placing code close to the legal text (Merigoux et al., 2021, secs. 2–4). It is useful when nested qualifications would otherwise become a long, undocumented sequence of `if` statements.

For a small banking illustration, take the product-explanation relief in Annex 1 paragraph 10.3. Within the selected solicited-transaction profile, an explanation remains required when the client requests it or raises material queries. A local representation can say:

Listing 2: An exception with an explicit guard. The combined guard avoids creating two competing exceptions for the same requirement.

```
normally: product_explanation_required = false
exception when client_requests_explanation
            OR client_has_material_queries:
    product_explanation_required = true
always retain: current_offering_documents_required = true
```

The *guard* is the condition after “when”. If the guard is true, use the exception; if false, use the general value. If evidence about the guard is missing, LegalMath leaves the requirement unresolved. The separate offering- document requirement is not inside the explanation exception. A programmer who implements one switch for both has changed the model (SFC and HKMA, 2023b, Annex 1, para. 10.3).

There is an important distinction between borrowing this organization and claiming to implement Catala itself. Catala’s core calculus distinguishes an empty applicable value and a conflict between exceptions. LegalMath additionally needs incomplete bank evidence to remain explicitly unknown. Its proposed default operator evaluates every sibling guard: two established exceptions conflict; one established exception with an unresolved competing guard remains unresolved; one established exception with all competitors refuted supplies the value; all refuted guards use the general value. Nesting expresses an adopted priority. File order does not.

Suppose two exceptions have been written, one for a request and another for a material query. A client who does both would activate both. Under the proposed sibling-conflict semantics, identical resulting values still conflict. Combining the two conditions into one guard expresses the intended single exception. That example shows why choosing a language is a substantive design decision: a rule engine’s treatment of overlapping rules must match the representation the reviewer has adopted. The delivered Java example uses the equivalent explicit Boolean condition for this one requirement; its compiler rejects the general default operator until that operator has its own implementation tests.

2.3 Stipula: agreements, authorized actions and state changes

A wealth calculation is a function of facts. An agreement also has a history. Before signature, after signature, after withdrawal and after an amendment, the same attempted action can have different consequences. Stipula is a language for expressing those interactions. The requested 2021 paper defines parties, agreement terms, fields, assets, states, guarded functions and scheduled events (Crafa et al., 2021, secs. 3–4). The original language in that paper is type-free; the typed decision representation proposed here is a separate design.

A *party* is an actor allowed to participate, such as a lender or borrower in the paper’s rental example. An *agreement* establishes the parties and the terms on which they agree. A *field* stores information, such as a price or agreed period. An *asset* represents a transferable resource. A *state* records which stage the agreement has reached. A *function* specifies an action a named party may invoke in a particular state; a guard adds a condition for the action. Its body changes fields,

sends information or transfers assets, and its transition identifies the next state. A scheduled *event* specifies an action to attempt at a future time if the agreement is then in the required state.

For example, the rental contract in the paper first records an offer. The borrower accepts by supplying the required payment, which moves the contract into its using state. The contract provides the use information and schedules an event for the end of the rental period. Either an appropriate borrower action or the scheduled event can finish the interaction and release the held payment. The language gives a precise account of which party can trigger each step and when the payment moves. Assets have special transfer behavior; they are not ordinary numeric fields that an assignment may freely duplicate (Crafa et al., 2021, sec. 3, rental example and grammar).

The banking analogue uses the state and actor ideas but does not transfer client assets. The following is explanatory pseudocode, deliberately distinguished from compilable Stipula syntax:

Listing 3: A consent model inspired by Stipula. “Record” observes an event; “allow” describes the control’s decision.

```
parties: Client, Bank
terms: client_id, product_category, threshold, monitoring_mode
states: NOT_ESTABLISHED, ACTIVE, WITHDRAWN

NOT_ESTABLISHED:
  record Client.written_agreement(terms, evidence)
  when Bank.confirmed_required_explanations(evidence)
  -> ACTIVE

ACTIVE:
  record Client.withdrawal(instruction_id) -> WITHDRAWN
  allow Bank.apply_streamlining(transaction)
  when category_matches(transaction, terms)
    AND all_other_required_conditions_hold(transaction)

WITHDRAWN:
  do not allow Bank.apply_streamlining(transaction)
  record a new, fully evidenced agreement -> ACTIVE
```

The terms bind consent to a client, category and arrangement. A new agreement after withdrawal is a new generation of consent, not deletion of the withdrawal. The regulatory duties themselves come from the applicable regulatory sources. Client and bank agreement can establish the consent contemplated by those sources; it cannot amend an SFC requirement. This distinction is essential when borrowing a contract language for a regulatory-control system. The figure makes those changes visible without requiring the reader to know a particular language’s punctuation.

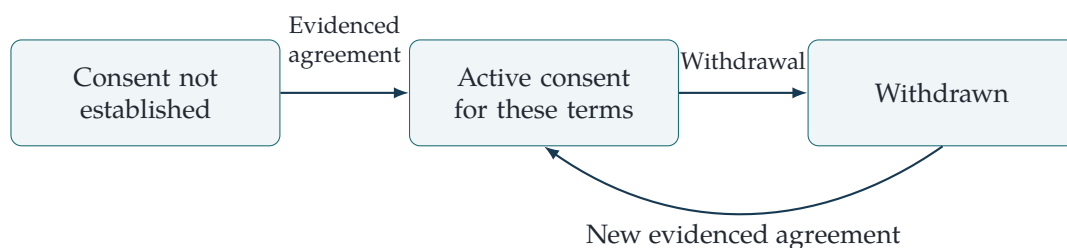


Figure 2: Consent transitions for the example. An incomplete event history produces an unresolved assessment outside these established states. It is not evidence of an active agreement.

Execution requires a state, stored terms and a history of ordered events. In Stipula’s operational semantics, the runtime also maintains pending scheduled events and a clock. In the inspected 2021 semantics, due events take priority over function calls at the same time. That is a language rule, not a rule of Hong Kong banking law. LegalMath must adopt an explicit ordering policy for withdrawal and order events, record the authority for their sequence, and retain an unresolved result where the ordering cannot be established. It cannot import Stipula’s clock convention as if the circular prescribed it.

There is a second adaptation. A control may prohibit streamlined treatment after withdrawal, but the bank might nevertheless have processed a transaction. Deleting that transaction from the history because it is an illegal transition would hide the very incident a compliance system needs to detect. LegalMath therefore separates the observed event record from the decision about whether the proposed action is permitted by the selected control. Normative languages such as eFLINT and Symboleo help make this distinction explicit: a duty or violation is represented separately from the fact that an action happened (van Binsbergen et al., 2020; Sharifi et al., 2020).

2.4 What it means to verify such a model

A property is a precise question about the model. For the consent example: “After a known withdrawal, and before a new qualifying agreement, this control never returns that the streamlining conditions are met.” Testing checks selected histories. Model checking explores all histories within a specified finite model. A mathematical proof establishes the property under explicit assumptions. None establishes that a real client signed a document, or that the selected provisions are the complete legal regime.

Stipula’s behavioral-equivalence work compares the observable interactions of two formal contracts. Two implementations can use different internal state names yet be equivalent if the relevant party actions, communications and asset behavior match. This is useful for comparing formal implementations. It is not a test that the English circular and the contract mean the same thing (Crafa et al., 2021, sec. 5).

The Stipula–KeY work supplies a particularly relevant connection to Java. Its translator builds Java verification models from a supported class of Stipula contracts and adds specifications in the Java Modeling Language (JML). KeY reasons about Java programs and these specifications. A JML precondition states what must hold before a method; a postcondition states what the method must establish; an invariant states a condition preserved at the designated program points. For instance, an illustrative postcondition can require a withdrawal method to leave consent inactive, while an invariant can prevent a successful streamlining decision in that state. The proof obligation concerns all executions covered by the model and its assumptions (Hähnle et al., 2025; Stipula contributors, 2026b, secs. 3–5).

That translation is a useful research route, but its Java verification model is not automatically a deployable bank integration. The paper restricts cycle structure and abstracts some event conditions; the adapter still has to preserve timestamps, evidence and the bank’s actual interface. The concrete Java demonstration in this proposal instead compiles the stated decision subset directly. It compares Java with a separately implemented reference evaluator and source-derived examples. A later KeY or Lean task can strengthen the compiler evidence for a chosen property without changing who approves the legal interpretation.

2.5 Solvers, explanations and search in ordinary programming terms

A satisfiability solver searches for values that make a set of logical constraints true. To compare the correct wealth alternative with a mistaken conjunction, give it both formulas and ask for inputs where their answers differ. HK\$35 million and HK\$90 million are one such counterexample. An SMT solver extends this idea with theories such as exact integer arithmetic. It is useful here because money can be encoded as integers. Before treating “no counterexample” as evidence, the system must establish that the allowed input domain is nonempty and that both formulas were encoded faithfully. A timeout means that the search did not finish.

A justification engine answers a related question: which rules and facts support a conclusion? In a rule program, `eligible(C) :- financial(C), consent(C)` means that both supporting predicates are needed to derive eligibility for the same client *C*. `s(CASP)`, used by `s(LAW)`, can expose the supporting derivation and assumptions rather than returning only a label. An assumed consent predicate in a hypothetical answer is still an assumption; it must never be written back as an observed agreement (Arias et al., 2024).

Search over interpretations sits above these calculations. An agent may propose two readings of a clause, retrieve a referenced definition, and ask a solver for a case that separates them. A Monte Carlo tree or UCB scheduler can choose which investigation to try next. Its score measures usefulness under the chosen search objective; it does not certify the resulting legal reading. The adopted meaning must come from source-based adjudication. The existing MCP projects can assist these bounded jobs, as explained in Section 7.

3 A complete worked change: circular 23EC35 to a Java decision

The running source is the joint SFC–HKMA circular of 28 July 2023, *Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors*. Its main text introduces the approach; Annex 1 sets out the guidance; Annex 2 answers implementation questions. This example follows those documents through collection, interpretation, evidence mapping, evaluation, Java generation and an operational change. The facts about the client are invented for the demonstration. The regulatory provisions are identified individually so that those two sources of information remain distinguishable (SFC and HKMA, 2023a,b,c).

3.1 The business question and the facts the bank would need

At 10:00 Hong Kong time on 21 September 2026, an adviser proposes a solicited transaction in an ordinary corporate-bond category for Ms Lee, a synthetic individual client. The question is whether the bank may apply the selected streamlining treatment to this transaction, subject to its other controls. The example uses paragraph 8.3(a), which monitors gross exposure at execution. It does not use the designated-account alternative in paragraph 8.3(b).

Assume the bank has already assessed Ms Lee as an individual professional investor under the applicable imported definitions. Her qualifying portfolio is HK\$40 million: HK\$35 million on her own account plus a documented 25 percent share of a HK\$20 million joint portfolio with a non-associate. Her total assets are HK\$120 million, liabilities HK\$15 million and primary residence HK\$30 million, giving HK\$75 million of net assets excluding the residence. The portfolio route meets paragraph 3.1 exactly; the net-asset route does not. The ownership agreement, valuations and assessment would be separate evidence records in the bank. The example’s normalized inputs already incorporate them.

Ms Lee has no relevant degree, professional qualification or qualifying year of work experience

in this example. She instead has five transactions in the chosen category, dated 8 January and 12 June 2024, 14 February and 7 November 2025, and 20 August 2026. These dates lie comfortably inside the relevant three-year window; the example does not settle a disputed endpoint convention. The bank has separately recorded its reasonable satisfaction about her sophistication and assessed her objectives as non-conservative. Five transactions are evidence for a specified route, not an automatic substitute for that assessment (SFC and HKMA, 2023b, Annex 1, paras. 4.1, 5.1, 7.3).

The client has selected this category, received the category information and required explanations, and given the prior written acknowledgment for the arrangement. The agreed streamlining threshold is HK\$10 million. Existing gross exposure is HK\$8 million; the proposed transaction adds HK\$2 million, including leverage, bringing the monitored exposure to HK\$10 million. The control compares that gross amount with the threshold, not the cash margin posted for the purchase. The threshold rationale and assessment are retained; the example treats the annual review and reminders as current (SFC and HKMA, 2023b,c, Annex 1, paras. 6–8, 12–14; Annex 2, FAQ 7).

Finally, current offering documents have been delivered. There is no unanswered request for product explanation or material query. The transaction has been assessed for the outsize or material-transaction warning requirement; this synthetic case does not require that warning. These are retained controls. They do not disappear because the wealth condition passed (SFC and HKMA, 2023b, Annex 1, paras. 8.4, 10.3).

The facts are intentionally close to two boundaries. A one-cent reduction in qualifying portfolio, with the other facts fixed, defeats the financial condition. A one-cent increase in projected gross exposure defeats the selected execution-monitoring condition. A test suite containing only a very wealthy client buying a very small position would miss both defects.

3.2 Collect the whole instruction before extracting rules

The intake record contains reference number 23EC35, the public document URL, issuer, publication date, language, retrieval time and content digest. It links the main text to both annexes and records their digests separately. The annex PDFs are retained as immutable bytes. Extracted text is a derivative, with page breaks, its own digest and exact character offsets. A source anchor therefore identifies a particular passage in a particular preserved edition, even if the public web page later changes.

For example, the financial condition is anchored to Annex 1 paragraph 3.1. The supplied source record includes the PDF digest beginning f83719981d74, the derivative digest, the page and the span digest. The validator checks those against the actual retained files. Changing one character in the extraction invalidates an old offset-based anchor until the passage is matched and reviewed again. A language model cannot repair an anchor by supplying a plausible-looking quotation.

Collection also produces a dependency list. Annex 1 imports definitions from the Professional Investor Rules and the Code of Conduct; the opening footnote identifies suitability requirements in Code paragraphs 5.2 and 5.5(a). Annex 2 and Annex 1's footnotes refer to further product guidance. The demonstration accepts reviewed input assessments for these imported matters. A production release must name the adopted source editions, affected legal entity and business activity. The synthetic bundle's September 2026 validity interval is a test setting, not a finding about the circular's legal commencement or the continued applicability of every imported provision.

An agent should next produce a provision inventory, before proposing a formula. The following

inventory covers the numbered guidance and FAQ families. A row can lead to a decision rule, an evidence calculation, a human assessment or a continuing task. “Outside this profile” means a separate implementation is needed; it never means the passage was irrelevant and discarded.

Table 1: Disposition of the complete circular for the worked profile.
Subparagraphs within each stated range remain part of that row.

Source	Substantive instruction	Treatment in the worked change
Main text; Annex 1 opening and footnote 1 1–2	Introduce streamlining, retain conduct and risk controls, and import definitions and suitability requirements. Individual criteria at the relevant date; separate investment-holding corporation route.	Record authority, perimeter and dependencies. The Java result covers selected conditions only; the bank retains its wider control chain. Individual PI assessment is an input. A corporate client returns outside profile; ownership and qualifying-owner aggregation require a separate rule family.
3.1(a)–(b)	Inclusive alternative wealth thresholds and currency equivalents.	Execute exact HK-cent comparisons joined by OR. Foreign currency requires an adopted conversion mapping before evaluation.
3.2(a)–(d), 3.3, 3.4(a)(i)–(iii), (b)	Portfolio and asset/liability attribution; residence exclusion; written or equal joint shares under the stated circumstances.	Reviewed normalization produces portfolio and net assets. A missing agreement is unresolved evidence, not established absence. Preserve each ownership calculation.
4.1(a)–(d), 5.1	Reasonable sophistication assessment, alternative supporting routes and non-conservative objectives.	Execute the alternative routes and require the dated assessment and objectives. Do not substitute the representative’s attributes.
6, 7.1–7.3	Restrict transactions to selected categories and threshold; define categories, provide information, and apply category qualification.	Require selected category, resolved category explanations and information. Count same-category transactions or establish another qualification route.
8.1–8.2	Client specifies a threshold appropriate to circumstances; bank retains the setting and rationale.	Accept an agreed, reviewed absolute HKD threshold and rationale. A percentage-of-AUM arrangement needs a versioned conversion to the same input.
8.3(a)	Monitor gross exposure on execution.	Compare projected exposure including the candidate transaction with the agreed limit; host must reserve exposure atomically before using the decision.
8.3(b)	Alternative monitoring for designated accounts and new funds.	Explicitly outside profile. Implement its own account, transferred-position and top-up events before enabling it.
8.4–8.5	Detect outsize or material transactions and warn; review threshold compliance annually and alert if exceeded.	Require transaction assessment and conditional warning. Schedule review/alert tasks; a reviewed composite reports annual controls current.
9; 10.1–10.4	Define solicited-transaction relief, with explanation exceptions and offering documents retained.	Execute the solicited profile. The result identifies applicable relief; it does not disable every suitability or disclosure control.

Source	Substantive instruction	Treatment in the worked change
11.1–11.5; footnotes 2–4	Different relief for unsolicited complex-product transactions, including document alternatives and warning arrangements.	Outside profile. Preserve the distinct document/evidence branches, bond information alternatives and annual-warning requirement for a later implementation.
12.1–12.3	Reasonable satisfaction, written assessment and correspondence recording category and threshold choices.	Dated assessments and document references feed named facts; records remain independently inspectable.
13.1(a)–(d), 13.2	Prior written agreement and explanation of consequences, categories, threshold and withdrawal/amendment rights.	A checklist-backed acknowledgment input plus a replayed consent state. Every item in the checklist must be evidenced; a signature alone is insufficient.
14.1–14.2	Annual continued-qualification/agreement review and written reminders covering consequences, categories, threshold, breaches and rights.	Ongoing tasks and reviewed status. Suspending streamlining when review is overdue is a proposed additional bank restriction.
Annex 2, FAQ 1	Appropriate reliance on disclosures after reasonable KYC efforts; clarify inconsistency.	Define admissible evidence and escalation; do not invent a blanket requirement for independent third-party confirmation.
FAQ 2	Client and power-of-attorney distinctions.	Attribute qualifications to the client. Apply the FAQ to transaction-history evidence without copying the attorney's sophistication.
FAQs 3–4, including notes	Category design, difficult product distinctions and category information.	Bank-owned category taxonomy, versioned mappings and information checklist; uncertain mapping requests review.
FAQ 5	Threshold should reflect the client's circumstances; bank-held AUM may be a limited part of wealth.	Recorded client circumstances and threshold rationale. No universal percentage is invented.
FAQ 6	Explain both monitoring approaches, including designated-account transfers, appreciation and new funds.	Use execution monitoring here. Keep alternative-account rules visible; do not infer forced unwinding merely from exceeding its threshold.
FAQ 7	Gross exposure includes leverage.	Feed leverage-inclusive projected exposure to the execution comparison.

This inventory is the first useful review deliverable. It exposes omissions before a test generator starts exploring the rules that happened to be encoded. The main text, annexes and incorporated definitions are the review boundary; the numeric threshold is only one part of it.

3.3 Record the interpretation and who supplies each fact

The agent's draft should separate source statements from proposed bank choices. For paragraph 3.1 it records an inclusive alternative and links both comparisons to the paragraph. For paragraph 13 it records an arrangement-specific written acknowledgment and the withdrawal right. For paragraph 14 it records the annual duties. It then proposes, separately, that the bank suspend use of the streamlined route when its annual-review status is not current. A compliance reviewer can accept, amend or reject that last proposal without losing the source duty.

Two unresolved issues are deliberately visible. First, the bank must settle ownership, currency and valuation mappings for wealth and exposure. Second, the calendar conventions for transaction windows and annual deadlines must be adopted. A complete source-based interpretation may still rely on human assessment: “reasonably satisfied” and “material transaction” do not acquire objective numerical definitions simply because the system needs an input.

In the production workflow, the preparer submits a specific bundle digest and supporting examples. An authorized reviewer other than the preparer accepts that exact version, with the perimeter and unresolved issues visible. Editing a rule changes the digest and makes the previous acceptance insufficient for release. The delivered example carries DEMONSTRATION_ONLY throughout: its synthetic assessments and proposed bank choices have no invented reviewer approval. Draft evaluation is useful before approval; production eligibility requires the release controls described later.

Table 2: Input contracts for the demonstrated Java profile. A known value always has evidence, a validity interval and a recording time.

Input or group	Meaning and intended owner
Profile flags	profile_individual, profile_solicited, profile_execution_monitoring: the host’s classified client, activity and monitoring mode. False selects an unsupported profile; unknown requests review.
individual_pi	Compliance assessment under the adopted Professional Investor Rules and Code definitions, for this legal client and date. It is distinct from merely being an individual.
portfolio; net_assets_ex_home	Valuation/operations supplies exact HK cents after reviewed ownership, liabilities, residence and currency treatment. Preserve the calculation inputs and source evidence.
Qualification fields	relevant_degree, professional_qualification, relevant_year_of_work; alternatively category_transactions_3y. Client-level evidence is mapped to the selected category and relevant period.
Assessment fields	reasonable_sophistication_assessment, non_conservative_objectives, written_assessment_retained. Compliance/relationship management supplies the assessment and records; missing review is unknown.
Category fields	category_selected, category_information_delivered, category_explanation_resolved. Client choice and delivery evidence must match the transaction’s versioned category.
Acknowledgment and consent	prior_written_acknowledgment_complete covers every required item in 13.1–13.2; active_consent comes from the client/category consent history. Document completeness and present consent are different facts.
Threshold and exposure	streamlining_threshold, projected_gross_exposure are HK cents; threshold_rationale_retained is an assessment/record check. Order management supplies a consistent exposure snapshot including leverage.
annual_review_current	Reviewed composite covering continued qualification/agreement, threshold review, required reminders and breach alerts. The demo consumes this status; it does not calculate annual deadlines or send reminders.
Product information	current_offering_documents_delivered, explanation_requested_or_material_query, explanation_delivered. Delivery/interaction records determine whether the remaining request has been answered.

Input or group	Meaning and intended owner
Transaction warnings	transaction_materiality_assessed, outsize_or_material_transaction, material_warning_delivered. The assessment is required; a warning is required if the assessment says so.

Composite inputs are integration boundaries, not permission to hide missing work. For example, the acknowledgment evidence record must link the prior written agreement, qualification explanation, chosen categories and threshold, consequences and withdrawal/amendment explanations. A rejected or incomplete checklist makes the composite false or unknown under the bank's adopted evidence policy. The reviewer can open the constituent items. The generated decision library does not authenticate documents or decide whether an explanation was adequate.

3.4 Construct and evaluate the executable specification

The worked bundle has ten named Boolean rules. The financial rule performs the two monetary comparisons. The category-qualification rule joins the four supporting routes. The assessment rule combines individual PI status, finance, category qualification, reasonable satisfaction, objectives and written records. A separate review-policy rule records the proposed bank restriction under its own bank-policy interpretation. The arrangement rule checks category choice and information, acknowledgment, active consent, threshold rationale and the proposed annual-review restriction. The threshold rule compares limit with projected gross exposure. The explanation and warning rules express conditional retained actions. The retained-information rule combines these with delivery of current offering documents. The root rule requires assessment, arrangement, threshold and retained information within the selected profile.

Every reference points to a named, unconditional subrule; the root alone selects the profile. The graph is acyclic, so evaluation terminates after a finite number of expressions. Money and counts are exact integers. Unknown evidence is propagated by the specified three-valued operations; contradictory supplied records return a conflict before evaluation. The compiler type-checks the tree and rejects unsupported operators instead of silently choosing a different meaning.

For Ms Lee, the trace reads as follows. The portfolio comparison is true at HK\$40 million; the net-asset comparison is false at HK\$75 million; finance is therefore true. Five category transactions establish the qualification route. The other reviewed assessment and arrangement conditions are true. HK\$10 million is at most the agreed HK\$10 million, so threshold monitoring succeeds. The explanation request is false, the transaction is assessed as not requiring the material-transaction warning, and offering documents are delivered. The root returns STREAMLINING_CONDITIONS_MET. The result includes the rule trace, source-anchor identifiers, rule-bundle and fact-snapshot digests, both evaluation timestamps and a digest of the result itself.

This successful conclusion remains narrow. It identifies the selected streamlining conditions under the supplied assessments and proposed review restriction. The host must still apply other relevant product, credit, market, financial-crime, conduct and order-execution controls. Its own release process decides which approved control version is usable. A demonstration result is never automatically promoted by setting a Boolean input named "approved".

Table 3: Distinguishing cases derived from the running interpretation. Other inputs remain as in the successful synthetic case unless stated.

Changed fact	Result	Why it changes the operational response
Portfolio HK\$39,999,999.99; net assets HK\$75m	Standard process	Both financial alternatives are refuted.
Portfolio HK\$35m; net assets HK\$90m	Conditions met	The alternative net-asset route suffices.
Portfolio missing; net assets HK\$90m	Conditions met	Missing portfolio is reported, but does not block the established alternative.
Portfolio missing; net assets HK\$70m	Review required	Portfolio evidence can change the answer. It is a blocking input.
Conservative objectives	Standard process	Wealth does not overcome paragraph 5.1.
Four category transactions; no other qualifying route	Standard process	The countable route is not established.
Chosen category does not match	Standard process	General SPI attributes do not extend the selected categories.
Gross exposure HK\$10,000,000.01	Standard process	The execution-monitoring condition is exceeded.
Written acknowledgment incomplete, or consent withdrawn	Standard process	Prior documentation and current consent are both required.
Client requests explanation; no answer recorded	Standard process	Paragraph 10.3 retains the requested explanation.
Material transaction; no warning delivered	Standard process	The applicable paragraph 8.4 warning remains outstanding.
Two inconsistent portfolio records	Review required	An evidence conflict is not resolved by choosing the more convenient amount.
Designated-account monitoring selected	Outside profile	Its different paragraph 8.3(b)/FAQ 6 logic has not been executed by this library.

The associated actions are equally concrete. A missing portfolio requests the valuation and ownership evidence. An unanswered explanation routes a task to the adviser and retains the interaction record. An exceeded execution threshold routes the order to the standard process or a separately reviewed change in arrangement. An evidence conflict routes to its data owner. These are proposals for the host workflow, not a generated instruction to place or cancel a trade.

3.5 Generate, build and call the Java release candidate

The demonstrator stores the typed tree in `spec/spi-control.bundle.json` under `examples/java-dry-run/`. A deterministic compiler walks each node and emits a call to its corresponding Java operation. An integer literal becomes a `BigInteger`; a fact reference becomes a typed fact lookup; any becomes the explicitly three-valued alternative; a rule reference becomes a memoized Java method. Source identifiers and the bundle digest are compiled into the class. There is no language-model call, network retrieval or rule-file parsing during a transaction decision.

The generated class and a small support library are compiled into `spi-controls-demo.jar`. A separate host example is then compiled against that JAR. This tests the actual deployment boundary: the caller sees the public library API and receives a structured decision. Section 6

specifies the API and how a bank adapter must prepare its inputs. The same build compares complete Java decision results with the independent Python reference, rather than merely checking that compilation succeeded.

Three deliberate generated-code mutations make the test useful. Changing the portfolio comparison to strict rejects the inclusive boundary. Replacing the wealth alternative with a conjunction rejects the alternative route. Bypassing the consent condition permits the withdrawal case. The build compiles and runs each mutated rule class against the same client harness and requires at least one independently specified status to fail for each mutation. Matching hashes alone are not considered detection of a semantic defect.

3.6 Withdraw consent, replay the decision, then change the rule

At 10:05 Hong Kong time, suppose Ms Lee withdraws. The event is recorded at 10:06. A replay has two timestamps: the time being assessed and the cutoff for what the system knew. The 10:00 decision remains reproducible with its original facts and knowledge cutoff. A decision at 10:10 using a complete history known at 10:10 sees the withdrawal and returns the standard-process result. The Java host demonstration performs this sequence by replaying the consent events, replacing the consent fact with the evidenced inactive value, and evaluating the same generated control again.

If the event feed is incomplete, absence of a withdrawal record cannot establish consent. The replay interface therefore requires a completeness assertion for the relevant history window and the time that assertion became known. A record cannot certify completeness through a future instant; the replay rejects that premise as unresolved. In the late-arrival case, a history complete only through 10:04 does not support a 10:10 conclusion, even if the last visible event is a grant. Identical event identifiers and payloads are idempotent duplicates; differing payloads under the same identifier conflict. Events at the same occurrence time require an authoritative sequence. The demo covers these behaviors explicitly; a production adapter must establish the completeness and sequencing assertions, not invent them.

Now consider a later amendment. As an engineering test only, suppose an approved rule version changes a threshold or retained information condition. This is not a claim that the SFC has issued that change. The collector creates a new source revision; the workbench marks affected definitions, rules, examples and Java classes. The reviewer sees the semantic difference and the clients whose synthetic outcomes change. The compiler creates a new bundle digest, generated source and JAR. Release approval applies to those exact versions. The old bundle, JAR and evidence remain available for replay; the bank changes the active release atomically at the agreed effective instant.

Rollback is a software-release action, not reversal of the legal calendar. An operationally defective JAR may be replaced with another implementation of the same approved legal version. Restoring an older legal rule after its applicability has ended would require a separate, valid legal basis. Historical replay selects the version that applied to the historical question, rather than whichever JAR happens to be active today.

The dry run thus reaches an actual executable Java boundary while exposing the work that remains: bank adjudication of interpretation, imported definitions, data and calendar mappings, the reviewer application, and production integration. The product plan below turns those remaining activities into testable modules.

4 What the literature contributes

Computational law has developed several answers to different parts of this problem. Logic programming makes statutory dependencies executable. Languages such as Catala organize definitions and exceptions close to legal text. Normative languages represent duties, permissions and violations. Contract languages model interactions through time. Recent language-model research attempts to cross the remaining boundary from prose to these representations. A useful product combines these contributions selectively, because their guarantees concern different objects.

4.1 Scope and method of this review

The review began with the seven requested links and expanded through their references, forward-citation searches for Catala and Stipula, and searches on executable law, legal reasoning, formalization, dates, testing and process compliance. ResearchAssistant supplied discovery records and local PDF extraction. The saved OpenAlex citation searches returned 52 entries for the selected Catala record and 13 for the selected Stipula journal record. Those are retrieval counts for particular indexed editions, not complete or comparable measures of influence. Preprints, revised editions and new publications can be missing or duplicated.

The local library contains 38 paper editions representing 37 distinct works. The AAI tax-reasoning paper and its extended preprint are retained separately; all seven requested sources are present. Technical reading covered the relevant methods, semantics, evaluation and supporting appendix material. The accompanying notes identify the inspected sections, version differences, implementation checks and limits of adoption (LegalMath research notes, 2026a). This is a focused technical review supporting a product decision, rather than a claim to have enumerated every citation. Unavailable secondary leads are recorded separately and do not support implementation claims.

The expanded search treats eight families separately: executable statutes; defaults and defeasible priorities; obligations and violations; temporal process compliance; authority and provenance; legal languages, compilers and testing; neurosymbolic formalization and evaluation; and deployable rule engines. Important foundations predate Catala, while contract languages have partially separate citation communities. The retained forward-citation records have individual dispositions in docs/papers/coverage.md; additional searches and backward references supply the missing mechanisms. Selection favors primary sources that define a needed mechanism or report an informative failure. Downloading a paper or screening its abstract is distinguished from inspecting its technical method. This is a bounded review of the prototype's design questions. Remaining retrieval gaps are stated rather than counted as read papers.

Version control is particularly important here. The ARc paper was first posted in 2025, but the inspected second version is from July 2026. The eFLINT paper's inspected version is from August 2026. The Stipula preprint requested by the sponsor has three authors; the later journal edition has a different author list. Bibliographic records and local filenames make these distinctions explicit. Published results are reported as their authors' findings; none of their benchmarks or proof developments was independently rerun for this proposal.

4.2 From statutory text to a language of definitions and exceptions

The British Nationality Act logic program is an early demonstration that selected legal provisions can be represented as executable dependencies. Its significance for this project is archi-

tectural: a conclusion can be explained through the conditions supporting it, while rules and facts are represented separately. The work also confronts negative information and temporal conditions, which remain central to bank data. It does not show that arbitrary legal prose has a unique automatic translation (Sergot et al., 1986).

Catala addresses a more specific obstacle: legal definitions often state a general rule and then exceptions to that rule. Its default calculus represents exceptions explicitly, distinguishes an absent applicable value from a conflict, and gives those situations operational meaning. In the core construction, multiple applicable exceptions can produce a conflict even when their computed values happen to agree. That is a deliberate semantics, not an invitation to resolve legal priorities by file order (Merigoux et al., 2021, secs. 3–4).

For LegalMath, this suggests a representation in which an exception names the rule it qualifies and preserves the source supporting its priority. A discount exception to a restriction on gifts should remain visibly attached to that restriction. Two independently imported exceptions that both apply should trigger a review of their relationship, not an arbitrary choice by the language model. The product should also distinguish this normative priority from a document’s publication date: a newer explanatory document does not mechanically override every older source it mentions.

The Catala compilation result is relevant but narrower than the phrase “verified legal software” might suggest. The paper proves properties of a translation from the core default calculus into a lambda calculus with exceptions, with a mechanized development in F*. It distinguishes that model from the separately implemented production compiler. The inspected repository’s formalization notes additionally disclose an admitted supporting lemma; this review has not replayed the development (Merigoux et al., 2021; Catala contributors, 2026, sec. 4). The useful precedent is to define a small semantics and state exactly which translation preserves it. It provides no theorem that a human or model has captured the meaning of an SFC circular.

Catala’s literate presentation also addresses the organizational problem. Huttner and Merigoux (2022, sec. 4.2) describe collaboration between legal experts and programmers around a common representation. The legal expert can challenge a condition without reverse-engineering application code, and the engineer can make an ambiguity concrete. The exploratory lawyer study in the original Catala paper gives useful interface evidence, but its small educational setting does not establish reliable defect detection by bank reviewers (Merigoux et al., 2021, sec. 6). LegalMath should borrow the collaboration model and test its own interface with the intended users.

Controlled language offers another way to make formal statements reviewable. Logical English retains a readable rule syntax while assigning a computational meaning to predicates and variables (Kowalski et al., 2023). For this product, a controlled paraphrase should be generated from the adopted rule, so changing the rule changes its explanation. It should display how variables are bound: “the client” must refer to the same person throughout a condition. Readability cannot substitute for a defined semantics, and unconstrained English should not be presented as if it were executable merely because it looks regular.

4.3 Why exceptions, duties and permissions need different logics

A threshold calculation asks whether a proposition follows from evidence. A duty asks what an actor ought to do in those circumstances. Treating both as ordinary implication loses the possibility of noncompliance: an instruction to conduct an annual review is not evidence that the review occurred. Input/output logic makes this separation explicit. A normative system transforms factual conditions into required outputs, without requiring every input to be an output or permitting unrestricted contraposition (Makinson and van der Torre, 2000, secs. 2–

3).

Let A contain factual propositions and let G contain pairs (a, x) expressing that condition a calls for outcome x . Writing Cn for classical consequence, the paper’s simplest operation is

$$\text{out}_1(G, A) = \text{Cn}(G(\text{Cn}(A))). \quad (1)$$

Here $G(S) = \{x \mid (a, x) \in G \text{ and } a \in S\}$ collects the required outputs whose conditions are in S . Thus the two uses of G denote the same normative system, first as a set of pairs and then as the operation of applying those pairs. First derive the input conditions, then apply the normative pairs, then unpack their consequences. Section 3.2 characterizes this operation by strengthening inputs, conjoining outputs and weakening outputs. A pair connecting “review due” with “review performed” supplies a required outcome; it does not create a recorded fact of performance. Other variants change disjunction and reuse. LegalMath borrows this separation rather than implementing all input/output logics.

Exceptions ask a different question: which applicable rule defeats another? Antoniou et al. (2000, sec. 2) distinguish facts, strict rules, defeasible rules, defeaters and an explicit superiority relation. Their proof theory records both definite and defeasible provability, and explicit failure to establish either. A defeasible conclusion needs supporting premises, failure of definite contrary proof and a response to every applicable opposing rule. Different supporting rules may defeat different opponents. This “team defeat” differs materially from selecting one table row or finding one Catala exception.

An unretrieved agreement is therefore not a defeater merely because the database has no row. The product must distinguish explicit negative evidence, a closed-world assumption and a reviewed default. LegalMath 0.1 chooses nested, source-linked defaults with conflicts between competing sibling exceptions. It does not implement the full defeasible proof theory. General priorities remain an extension requiring a stated defeat policy and conformance tests.

Permissions add another distinction. A weak permission is obtained when an opposite obligation cannot be established; a strong permission has explicit normative support. Governatori et al. (2012, secs. 2–3, 6) distinguish permissive rules from defeaters and distinguish obligation from factual fulfillment. Their algorithms transform a finite theory while preserving its defeasible extension. The soundness, completeness and linear-complexity results in section 5 concern that theory and specified data-structure assumptions, not arbitrary regulations. For the bank, failure to derive a prohibition must never by itself become a positive trade authorization.

A remedy is different again. The logic of violations of Governatori and Rotolo (2006, secs. 2–5) uses an ordered reparation connective: perform the primary duty; if it is violated, a further duty can become relevant. Replacing that structure with ordinary disjunction can make the remedy appear an equally acceptable way to satisfy the original requirement. A warning delivered after a deadline can be useful remediation without making the original control timely. LegalMath retains the breach assessment and records subsequent performance separately; general compensation and sanction reasoning remain outside its first event profile.

4.4 Dates and test generation expose defects that ordinary examples miss

The date-arithmetic work of Monat et al. (2024) demonstrates why even a simple phrase such as adding a month needs a convention. Calendar months have different lengths; leap days create boundary cases; sequences of date operations need not behave like addition of integers. The paper develops explicit rounding choices, formalizes date semantics and selected properties, and uses static analysis to detect ambiguities in legal computations. Its application to benefit rules connects the formalism to realistic source language rather than treating dates as a generic programming-library problem.

The immediate lesson for annual reviews and three-year transaction windows is to preserve the chosen calendar operation and its source or interpretation. “Three years” must not silently become a fixed number of days. A leap-day case can expose disagreement between two plausible implementations. The paper’s day-level formalization does not settle intraday questions such as withdrawal instructions arriving at a trading cut-off, nor does the mechanization make the whole static-analysis implementation verified (Monat et al., 2024, secs. 2–6). Those are distinct prototype obligations.

CUTECat complements this work by generating test cases from Catala programs using concolic execution: an execution carries concrete values and symbolic conditions, then a solver seeks inputs that take a different branch. The method specializes its exploration to default expressions and uses preferences to obtain inputs that are easier for humans to understand. In the reported evaluation, generated cases exposed 20 manually validated seeded mutations. A much larger example also shows that exhaustive-looking exploration can produce a substantial volume of cases and runtime (Goutagny et al., 2025, secs. 3–5).

In the running example, a concrete test with portfolio HK\$45 million takes the successful portfolio branch. The symbolic execution retains the condition $P \geq 40,000,000$ instead of forgetting how that result was reached. Negating it asks the solver for $P < 40,000,000$; holding the other financial route false yields a case that changes the overall answer. To distinguish inclusive from strict comparison specifically, request $P = 40,000,000$. Concolic exploration automates this alternation between a real execution and constraints describing another path. Mutation testing asks a different but complementary question: whether the examples notice an intentionally wrong operator. Both mechanisms are understandable and useful without importing the entire CUTECat execution environment.

LegalMath should borrow branch-directed and mutation-based testing, especially for inclusive boundaries, alternatives, exceptions and rounding. The result is evidence about the encoded program. A branch corresponding to an omitted paragraph does not exist for the test generator to discover. Independent source-derived scenarios are therefore essential. The official CUTECat artifact documentation was inspected, but its large execution image was not run; the prototype should first implement a small set of useful mutations before adopting that full toolchain (LegalMath research notes, 2026a).

4.5 Legal rules need authority, unknown facts and duties

LegalRuleML is relevant because it treats legal meaning as more than a Boolean expression. The standard addresses modalities, defeasibility, temporal aspects, sources and contextual information. Earlier work on legal interpretations shows how alternative readings and their contexts can be represented rather than collapsed prematurely into one rule (Athan et al., 2014; OASIS LegalRuleML Technical Committee, 2021, sec. 4). This is a strong basis for LegalMath’s metadata: every executable condition should identify its source, scope, adopted interpretation and relationship to other rules.

The standard is an interchange representation, however. XML that names an obligation does not by itself define the event processor that creates, satisfies or breaches it. LegalMath should adopt a deliberately small execution profile with explicit meaning, and use LegalRuleML concepts for interoperability. Requiring compliance officers to review raw XML would not solve the bank’s usability problem.

s(LAW) illustrates a different benefit of formal legal reasoning: explaining conclusions under alternative factual assumptions. Its use of s(CASP) combines logic-programming rules, explicit negative information and default negation, and produces justifications. The school-admission examples demonstrate discretionary conditions and hypothetical cases, including assumptions

that affect which conclusions can be derived (Arias et al., 2024, secs. 4–5).

This distinction matters when an SPI record lacks an agreement or a qualification. A hypothetical answer can say which additional fact would establish a condition. It cannot treat that fact as already established. LegalMath should display “would hold if this evidence were confirmed” separately from an evaluated client fact. Different treatments of negation must also remain explicit: absence from a database, a documented negative statement and a defeasible assumption are three different things. The s(LAW) examples support explanation and exploration; the linked implementation could not be fully audited in this review.

eFLINT contributes an especially useful operational distinction. A prohibited or otherwise impermissible action can still occur in the real world and have legal effects; the system must be able to record the resulting violation. A pure authorization engine that simply removes such transitions cannot audit what actually happened. The language’s duties, actions and open types are relevant to missing evidence and ongoing controls (van Binsbergen et al., 2026, secs. 3–5).

The 2026 eFLINT account also describes differences between implementations and versions. It notes the absence of a formal operational semantics and type system for version 4, and limitations concerning fine-grained explanations and normative priority; another implementation has a stable-model foundation. Guarantees must therefore be attributed to the actual version and engine used (van Binsbergen et al., 2026, secs. 6–7). The prototype should borrow the distinction between permission, occurrence and violation without assuming that installing an eFLINT interpreter supplies all the required proof evidence.

The original eFLINT paper makes the operational idea concrete through a transition-system interface: declarations establish institutional concepts, events change a configuration, and queries inspect permissions and duties. Action-compliance and duty-compliance are independent. Its GDPR example forms part of a small data-sharing/KYC exploration with ABN AMRO and ING, using separate policy and agreement specifications. This is relevant domain evidence, although it is a selective experimental case rather than a deployed private-bank control system (van Binsbergen et al., 2020, secs. 3–5). Operational connectors qualify observations into facts; the normative engine evaluates those facts; a reviewer owns the interpretation connecting them.

The stable-model implementation supplies a sharper warning about backend substitution. Esterhuysen et al. (2025b, secs. 4–7) encode states and transitions in Clingo and translate a restricted normal form of eFLINT clauses. The translation makes state explicit, normalizes aggregation and permits scenario search as well as checking a supplied scenario. Section 8 compares 108 specification–scenario pairs over 43 specifications. Agreement with the earlier interpreter is expected only where their semantics coincide; default reasoning is deliberately changed.

Their Example 9 shows how a prior implementation can retain a conclusion based on absence even after further facts invalidate that absence. Examples 10 and 11 distinguish no stable model from multiple stable models, while the older engine produces a particular result. A future ASP adapter that selects the first model silently would therefore choose an interpretation policy. LegalMath 0.1 avoids that problem by rejecting recursive definitions and defining unknowns directly. Richer semantics remain an option when the selected regulatory scope requires them. Pinning a package name alone does not pin the meaning of a rule.

To understand a stable model, consider the two rules “accept if rejection is not established” and “reject if acceptance is not established”. One self-consistent interpretation contains acceptance; another contains rejection. Selecting the first answer chooses between them without additional legal authority. In answer-set programming (ASP), a solver enumerates such stable interpretations after accounting for the program’s default-negation rules. Absence of a stable model can indicate inconsistent constraints, while several models can leave the answer dependent on the chosen interpretation. This miniature example explains the backend issue without equating

stable-model semantics with the three-valued fact evaluation used by LegalMath.

The accompanying eFLINT release makes these limits inspectable: it separates tests expected to agree from tests expected to differ, and documents excluded cases involving cyclic aggregation and unsatisfied models. The compiler's `Clingo.hs` also keeps action violations separate from duty violations. The source was inspected, not compiled or benchmarked. This supports a small differential test before adopting an engine, rather than extrapolation from the paper's overall claims (Esterhuyse et al., 2025a).

Symboleo contributes an explicit lifecycle for obligations and legal powers. A trigger creates a conditional obligation; its antecedent activates it; later events can fulfill, violate, suspend or terminate it. A power concerns the ability to change these legal positions, which differs from a duty to act. The initial paper gives statecharts, temporal predicates and selected axioms, with a sale-of-goods example checked using a Prolog prototype (Sharifi et al., 2020, secs. III–IV). It is preliminary feasibility evidence; the retrieved preprint does not contain all 27 referenced axioms. The official editor and model-checker repositories provide concrete examples and a `nuXmv` route, but were not executed (Smart Contract Modelling, University of Ottawa, 2026). The borrowing for LegalMath is to distinguish obligation creation, activation and performance. Suspension and legal powers require an extension to the MVP's simpler profile.

4.6 Stipula and the transition from a decision to a contract history

Stipula makes parties, agreement, state, assets, permitted functions and scheduled events explicit. Its semantics describes how a contract evolves when parties act and time advances. This is well aligned with consent, withdrawals, review events and requests for evidence. It also provides a notion of legal bisimulation, relating contracts through their observable interactions (Crafa et al., 2021, secs. 3–5).

The important borrowing is the state model, not a presumption that bank compliance requires a blockchain. The original paper discusses centralized as well as distributed execution. A bank can record a source-linked event history in its ordinary systems. Moreover, a similarity of formal interaction does not establish equivalence of the source texts as a matter of law. The accompanying discussion of legal calculi highlights how executable arrangements interact with external intervention and legal choices that code can otherwise suppress (Crafa et al., 2021; Crafa, 2022).

Several semantic choices need explicit review. In the studied Stipula semantics, scheduled events take precedence over function execution at the same time. That is a clear rule for the language; it is not an automatic answer to the bank's cut-off or consent-ordering questions. The prototype should construct a trace in which a withdrawal and a transaction have the same recorded time, and identify the finer ordering evidence or adopted policy required to decide it.

The Stipula-to-Java/JML work offers a concrete verification route. It translates contract structure into Java together with specifications and applies KeY to properties of the result. Four contract examples establish feasibility for the reported translation and properties. The inspected implementation includes a translator and generated examples (Hähnle et al., 2025; Stipula contributors, 2026b, secs. 3–4 and apps. A–B). This is worth a bounded experiment if workflow verification proves important.

The encoding also defines the limit of the evidence. Event conditions are abstracted through symbolic Boolean values rather than a complete implementation of banking calendars. Automated handling of the relevant loops uses inputs that remain constant across iterations. A proof about that model does not establish correctness for arbitrary changing portfolios, unbounded event queues or all real-time orderings. LegalMath must state and test any adapter's correspondence to its own event semantics before using the proof in a compliance package.

The 2026 reachability paper supplies a direct design constraint. Through counter-machine reductions it establishes undecidability for general and several restricted microStipula fragments. It also identifies a decidable fragment, called DI, with both disjoint function/event source states and immediate events, using a well-structured transition-system argument (Delzanno et al., 2026, secs. 2–4). DI is not the same condition as the disjoint-cycle construction in the Java/JML paper.

The product implication is to ask bounded, named questions. For example, can an approved rule set permit streamlined treatment after a recorded withdrawal in any trace of at most k events under a specified ordering model? A counterexample is useful immediately. Failure to find one supports only the checked bound unless an additional invariant proof establishes a broader claim. A timeout or unknown result must remain unresolved. The research does not imply that a small acyclic financial decision is intractable; it limits the promise of a universal verifier for unrestricted event-based contracts.

The amendment literature asks what happens to a running agreement when its code changes. Higher-order Stipula lets an amendment remove declarations, add declarations and run a transition, changing both program and current state. Restrictions can preserve parties, constant fields or asset conditions; an agreement predicate can require relevant parties to accept the change (Laneve et al., 2023, secs. 4–6). The retained source is the 2023 author manuscript *Legal Contracts Amending with Stipula*. It is not mislabeled as the different 2024 chapter *Programming Contract Amending*, whose full text was not successfully retrieved.

This poses a precise SFC transfer question. A new circular can change future decisions while existing consent arrangements or open duties need a separately justified transition. A replacement marker alone cannot decide that migration. The MVP creates an immutable new rule version and retains the version that opened each obligation. Any migration is an explicit reviewed operation. Runtime code replacement is unnecessary for the initial prototype and would make historical replay harder to explain.

The platform paper describes a visual editor, interpreter, type inference, unreachable-clause analysis and liquidity analysis (Laneve, 2026, secs. 3–7). Its liquidity property concerns resources that might remain trapped in a program. The underlying type analysis represents asset effects symbolically; cheaper sufficient tests differ from more precise bounded exploration (Laneve, 2023, secs. 4–5 and proof appendices). This is program-resource liquidity, not market or funding liquidity. The platform account assumes cooperative, strategyless behavior for the discussed property and states that the liquidity implementation is still being developed. Borrowing editor and explanation ideas is therefore a different commitment from claiming a production verifier for all bank workflows.

4.7 Consistency of clauses and compliance of processes

ContractCheck models legal contracts through structured clauses, an ontology and constraints, and uses automated reasoning to identify consistency and execution problems. Its share-purchase-agreement case study is particularly relevant to the presentation of contradictions: reviewers need to know which claims cannot coexist and which circumstances make an obligation executable (Khoja et al., 2025, secs. 3–7).

Two distinctions are essential when borrowing this approach. First, the study’s input formalization is manual; its reasoning results do not measure automatic translation fidelity. Second, showing that each claim can be executed under some circumstances differs from showing that all duties can be satisfied in one execution, and both differ from showing that every possible execution complies. LegalMath should label each of those questions separately. If a solver returns an unsatisfiable core, that identifies conflicting constraints; a smallest or minimal explanation

requires a further minimization procedure rather than an assumption about the core (Khoja et al., 2025, secs. 4–5, 8).

Process-compliance research reinforces the need to connect rules with activities, actors and the effects of execution. Ghose and Koliadis (2007) describe effect accumulation, constraint checking and repair in business processes. For the bank, the analogue is the difference between determining that a warning is required and showing that the relevant warning was delivered before the required event.

Tosatto et al. (2015, sec. 2) distinguish achievement obligations, which must be accomplished, maintenance obligations, which must hold across an interval, and punctual obligations. They also distinguish partial compliance, where some execution complies, from full compliance, where all executions do. Their complexity results concern specified process and obligation languages (Tosatto et al., 2015, secs. 3–4). The prototype should borrow those questions and delimit its model rather than import a complexity label for all regulation. The practical consequence is that a flowchart showing one successful path is insufficient evidence about the paths that omit a review or mishandle an exception.

Hashmi et al. (2016, sec. 3, Definitions 10–17) give a more detailed taxonomy. An obligation may apply at a point or persist across an interval; a persistent obligation may require an achievement once or maintenance throughout; achievement may or may not count before activation; and a violated obligation may persist or have specified compensation. A Boolean completed field cannot distinguish early action, timely performance, late performance, or a continuing control that failed briefly and recovered. The paper takes as input a function specifying which obligations are in force at each trace position. Supplying that function is a substantive modeling step, not something the taxonomy infers from prose. The MVP consequently supports one named profile, non-preemptive achievement, and explicitly rejects unsupported maintenance and compensation templates.

Governatori et al. (2016, secs. 3–4) connect LegalRuleML contexts and versions to annotated business-process tasks and a Regorous/PCL reasoner. Their procedure accumulates effects, derives obligations in force, carries pending obligations forward and checks fulfillment, violation and compensation. It distinguishes all-trace compliance from existence of a compliant trace, and conditions its reasoning claim on appropriate rules and complete annotations. Omitting a trigger can prevent an obligation from appearing at all, producing a deceptively clean result.

In the reported telecommunications pilot, roughly 100 paragraphs and associated definitions became 176 normative statements; initial legal mapping took about two weeks, followed by process-modeling workshops. These quantities show that formalization and operational mapping are real work; they do not estimate the time required by this bank. The proposal budgets compliance participation and counts preparation in every comparison instead of treating gold rules as free preprocessing.

4.8 Language models can propose formalizations; their inputs remain the weak point

The SARA tax-reasoning dataset makes explicit the ingredients an evaluation needs: statutory text, cases, formal rules and expected conclusions. Its corpus uses selected and simplified US tax provisions and constructed examples. It is a valuable experimental design, but neither a current tax authority nor a test set representative of SFC circulars (Holzenberger et al., 2020, secs. 2–3).

The later work connecting statutory reasoning to information extraction associates facts with spans in case descriptions and evaluates their effect on reasoning. This is a useful model for

recording where a client attribute came from, which person it describes and how it was normalized (Holzenberger and Durme, 2023, secs. 3–5). Its task-specific date imputations and semi-synthetic construction require caution: an extraction convention suitable for that dataset is not an evidential rule for missing bank records (Holzenberger and Durme, 2023, sec. 7 and apps. B, E).

ContractNLI adds a complementary annotation pattern. It separates supported, contradicted and not-mentioned hypotheses, with evidence spans in contracts. Its 607 non-disclosure agreements and fixed hypotheses expose the importance of exceptions and evidence spread across a document. It supplies useful ideas for an evidence-review interface, while evaluating document inference rather than compilation of regulatory rules (Koreeda and Manning, 2021, secs. 2–5).

Logic-LM and LINC both divide language interpretation from symbolic inference. Logic-LM routes formalized problems to solvers and uses error feedback to refine malformed programs. LINC translates into first-order logic and uses a theorem prover, with multiple samples and voting in its experiments (Pan et al., 2023, sec. 3) (Olausson et al., 2023, secs. 2–5). The transferable idea is a clear boundary: the model proposes the statement, and the solver reasons over that statement. Their benchmarks do not establish legal translation fidelity. Successful parsing can leave a wrong quantifier untouched, and repeated models can agree on the same missing exception.

The inspected Logic-LM repository also offers fallback strategies when formal execution fails, including a language-model answer or random selection for its benchmark setting. Those are not acceptable defaults for this prototype’s regulatory decisions. A failed formalization should produce an unresolved issue with a reason. Research code can be valuable while requiring a different operational policy (LegalMath research notes, 2026a).

The ARc paper is especially close to the proposed product. It separates creation and human vetting of reusable policy models from subsequent verification of natural-language material. Its workflow uses source passages, variable identification, repeated translations and deterministic explanations. The verification algorithm checks consistency and relationships between the policy, premises and claim, and distinguishes invalidity, validity, satisfiability and several forms of inability to decide (An et al., 2025, sec. 3 and Algorithm 1).

The method can be understood with three formulas. Let M be the reviewed policy model, P the premises extracted from a message and C its proposed conclusion. For example, the message may say that a client has a HK\$40 million qualifying portfolio and therefore satisfies the financial condition. The verifier first checks whether M allows the premises at all. If $M \wedge P$ has no possible assignment, the premises are impossible under that model; declaring the conclusion true by vacuous implication would be misleading. With consistent premises, if $M \wedge P \wedge C$ has no assignment, the conclusion is refuted. If $M \wedge P \wedge \neg C$ has no assignment, it is entailed. If both have assignments, the premises leave the conclusion undecided; the assignments show which extra facts distinguish the outcomes. For a purported complete SPI assessment based only on wealth, those missing facts could include objectives and knowledge. These checks reason within M ; they do not reconstruct an omitted circular paragraph.

ARc’s redundant-translation step addresses an earlier source of uncertainty. Several translations of the same message produce candidate premise/conclusion pairs. Algorithm 1 counts the fraction of translations that support a pair’s implication while not contradicting its premises. It does not merely compare the strings or require formula identity. A confidence threshold can make the system abstain and display competing readings. LegalMath can borrow that review interaction, but should preserve the candidate formulas and a distinguishing client example. Agreement among generated candidates is evidence of agreement, not independent proof that the interpretation is correct.

This is a strong precedent for an interpretation workspace with explicit abstention. It is also a warning about terminology. The paper’s reported “soundness” metric is $1 - \text{FP}/N$, an empirical rate over its decisions; it is not a theorem establishing the translator’s semantic soundness. At the 3/3 threshold in Table 1, 10 false positives among 1,689 decisions yield about 99.4 percent on that metric, while reported precision is 94.4 percent and valid recall is 14.9 percent. The 563 items were each run three times, so the total is not 1,689 independent policies (An et al., 2025, sec. 4, Table 1).

Those results motivate measuring both errors and abstention. A cautious system may produce few confident errors by deciding very little. Its business value then depends on whether the unresolved cases become easier to review. The ARc paper also discusses difficulties with cross-references, tables and implicit assumptions, all present in the bank’s problem (An et al., 2025, sec. 5 and app. A.1). Its semantic comparison of candidate translations should not be simplified into a claim that independent generation proves equivalence. ARc is a published commercial-system comparator, not an open-source implementation established by this review.

The requested AAI paper on trustworthy tax reasoning evaluates combinations of language models, facts and logic programs, including reviewed statutory rules and examples. It provides a particularly useful distinction for prototype design: reasoning with gold rules is a different task from generating the rules. The human work needed to prepare those rules and examples belongs in the comparison (Jurayj et al., 2026, 2025, Tables 2–3). The reported setting uses 100 numerical tax cases; its application-specific penalty for errors and abstention cannot be imported as a Hong Kong bank’s cost model.

In implementation terms, the tax method separates at least three jobs: encode the statute as a reusable logic program, extract the current case’s facts, and execute the program to obtain the requested amount. Supplying human-reviewed rules removes the first job from the model’s evaluation. Supplying formalized facts also removes much of the second. The corresponding LegalMath experiment must therefore hold the source packet and human preparation budget fixed, and report separately rule errors, fact-mapping errors and execution errors. A Java calculation can be exact while still using a net-asset input that includes the home; scoring only the final answer hides which stage needs repair.

The inspected companion code confirms that extraction and Prolog processing are separate stages and that execution failure can lead to abstention. LegalMath should adopt that separation and evaluate facts and rules independently. It should not quote a tax benchmark’s best configuration as evidence that the same method will reduce private-bank compliance errors. The reading notes also retain a discrepancy between the paper’s narrative attribution of a cost result and the configuration shown in its table; the proposal relies on the methodological distinction rather than that ranking (LegalMath research notes, 2026a).

The recent *Know Your Limits* preprint studies failures in legal reasoning and autoformalization, including assumptions that change the intended entailment task. Its examples strengthen the case for testing entity binding, implicit assumptions and distinctions between legal interpretation and strict entailment (Wang et al., 2026, secs. 3–5). It remains under-review evidence. Inconsistencies in the reported dataset denominators require clarification before quantitative reuse, and the choice of which ordinary or legal assumptions are admissible is itself a modeling decision. This proposal borrows its failure taxonomy, not a general error-rate estimate.

LegalBench broadens evaluation beyond a single reasoning dataset. Its tasks separate several kinds of legal reasoning; the evaluation appendix gives task-specific procedures. For rule application, legal reviewers check both correctness and whether an explanation contains the inferences connecting the facts to the result (Guha et al., 2023, sec. 3 and evaluation appendix). Repeating a circular accurately is not an explanation of its application to a client. Nor can a benchmark score become a banking acceptance criterion: the numerical SARA task permits er-

ror within ten percent, whereas one cent can change the intended threshold predicate. The limitations appendix identifies US-law, language, document-length and ambiguity gaps. LegalMath borrows independent answer guides and error analysis while constructing exact boundaries, source-dependency tests and amendment histories for its own task.

4.9 Software choices and the role of standards

The open-source landscape is sufficiently rich that a prototype need not build every component. It is not sufficiently uniform that selecting one package settles semantics, review, execution and proof at once. Table 4 identifies a practical role for each candidate. These are design recommendations based on the inspected papers and official documentation; installation, performance, security and integration have not been tested here.

Table 4: Candidate software and standards, with a deliberately bounded role.

Candidate	Contribution to LegalMath	Limit or prototype decision
Catala	Literate definitions, exceptions, executable decisions and verification-condition ideas.	Compare one adapter on the SPI decision fragment; do not assume event-workflow coverage or end-to-end verified compilation. (Catala contributors, 2026)
DMN 1.5; Apache KIE / Drools	Standard decision tables and Java-oriented decision execution.	Preserve hit policies, explicit unknown handling and explanations. Add a separate event model. (Object Management Group, 2024; Apache KIE contributors, 2026)
LegalRuleML	Source, authority, interpretation and normative metadata for interchange.	Adopt a small profile; a reasoner and execution semantics are separate components. (OASIS LegalRuleML Technical Committee, 2021)
Stipula; Stipula-KeY	Contract states, permissions, scheduled events and a Java/JML proof experiment.	Research adapter for a small workflow; document event abstraction and loop restrictions. (Stipula contributors, 2026a,b)
eFLINT	Normative simulation, duties and violations, including actions that occur despite a prohibition.	Borrow concepts and compare a bounded scenario; pin the implementation and semantics version. (van Binsbergen et al., 2026)
L4	Legal DSL, regulative rules, event traces, IDE and generated decision services.	Candidate for a later authoring/core comparison; documentation does not establish equivalence with RuleIR. (Legalese contributors, 2026)
Symboleo	Obligation/power lifecycles and a model-checking toolchain.	State-model alternative; preliminary paper case and software inspected, later journal paper remains a retrieval gap. (Sharifi et al., 2020; Smart Contract Modelling, University of Ottawa, 2026)

Candidate	Contribution to LegalMath	Limit or prototype decision
Regorous / PCL	Defeasible norms connected to annotated business processes.	Pilot evidence supports the method; deployed version, availability and license remain adoption questions. (Governatori et al., 2016)
Blawx / s(CASP)	Visual rule authoring, explanations and hypothetical reasoning.	Authoring experiment; Blawx describes its current scope as educational and experimental. (Lexpedite, 2026; Arias et al., 2024)
Z3; existing Lean route	Counterexamples, satisfiability, selected invariants and checked proof targets.	Specify supported theories and assumptions; preserve timeout and unknown outcomes. (Z3 contributors, 2026; LegalMath research notes, 2026b)
OpenFisca	Versioned computable rules and a tax/benefit microsimulation architecture.	Useful design analogue; the primary problem here includes conduct and workflow duties. (OpenFisca contributors, 2026)
Open Policy Agent	Rego policies evaluated against structured data; potential deployment adapter.	Its undefined-value behavior must be reconciled with the legal rule semantics. It supplies no natural-language legal interpretation. (Open Policy Agent contributors, 2026)
Accord Project / Cicero	Adjacent work on digital contracts and reusable contract technology.	Monitor for document/template integration; a fixed contract template differs from an amended regulatory corpus. (Accord Project, 2026)

L4 is a serious alternative to building the authoring environment from scratch. Its inspected repository exposes a parser, type checker, evaluator, IDE, decision service, generated schemas and graphical traces. Its regulative DEONTIC documentation names actor, action, deadline, successful continuation and breach continuation, and distinguishes MUST, MAY and SHANT (Legalese contributors, 2026). The 2023 L4 paper on deontics and time was identified but its full text was not retrieved, so no theorem from it is used here. Before adopting L4 as a core dependency, test exact money, missing evidence, simultaneous events and historical replay against the RuleIR cases. The proposal fixes that comparison target rather than making development wait for another language-selection exercise.

DMN deserves particular attention because its accessible tables can conceal a priority choice. Under a Unique hit policy, matching rows must be disjoint; Any permits overlaps with the same result; First selects by row order. Those policies are not interchangeable. A default exception tree imported from Catala cannot be exported as an arbitrary First table without checking that the meanings agree (Object Management Group, 2024, sec. 8.2.11). Review annotations also need their own source store: a visual note on a table is not automatically available as runtime provenance.

Licensing and reproducibility are separate from public availability. Catala, Apache KIE, OpenFisca, Z3 and the other public projects are candidates for reuse, but the precise pinned version, dependencies and intended distribution must be reviewed. A paper PDF, a hosted service and a public research repository carry different rights and maintenance expectations. The Stipula and tax prototypes should not be described as approved bank components merely because their source can be inspected. No third-party implementation has been imported into LegalMath as

part of this document work.

4.10 How to turn the citation network into a development program

The richest use of this literature is to connect each mechanism to a failure the bank can recognize. Catala and Logical English address inspectable definitions, while date semantics and CUTEat address difficult boundaries. LegalRuleML, s(LAW) and eFLINT address context, unknowns and obligations; Stipula and process-compliance work address histories. ContractCheck and SMT techniques address contradictory requirements, and the neurosymbolic papers address the fallible translation boundary. Each contribution can therefore be tested against a specific part of the bank's problem.

For each borrowed mechanism, the prototype should retain one source example, one SFC analogue, the assumptions that changed, and a test that would reveal a bad transfer. A Stipula event-ordering example becomes a consent-withdrawal trace; a Catala exception example becomes the fee-discount exception; an ARc ambiguity case becomes two proposed readings with a concrete client case on which they disagree. If the mechanism cannot represent the bank case without hidden conventions, that is evidence to revise the adapter or keep the case under human assessment. It is not a reason to force the circular into the language.

Citation expansion should continue during development around those unresolved questions. New papers enter the library with versioned full text and technical notes, and software enters through a small reproducible example. This preserves the benefits of a broad research base while keeping the product centered on reviewable regulatory change.

5 The product: a shared specification that can be reviewed and executed

LegalMath should be a regulatory specification workbench. Its primary users are the compliance officer interpreting a change, the product or operations owner deciding how it affects the business, and the engineer implementing the resulting control. An auditor or independent reviewer should be able to reconstruct the same decision later. The product succeeds when those people can inspect one maintained interpretation through views suited to their work.

The first release should support a narrow end-to-end task: take a selected public circular and its dependencies, prepare proposed rules, resolve or retain interpretation questions, evaluate synthetic cases, and export a versioned change package containing generated Java source, a compiled JAR and its evidence. Integration into bank systems follows the bank's ordinary change process.

5.1 What a reviewer should see

Opening the SPI financial condition should display the source passage and its annex context beside a controlled explanation: the portfolio route or the net-asset route can satisfy this particular condition. Definitions should be available in place, especially ownership attribution and exclusion of the primary residence. The same screen should show the boundary cases, missing evidence and where the condition is used in the larger SPI assessment.

The reviewer can change an interpretation, ask for a distinguishing example, or record a question for the policy owner. If two candidate readings differ only at the threshold, the workbench should produce that boundary case. If they differ because one applies to a representative and

another to the client, the example should name the roles. The user should not need to understand a solver formula to see why the choice matters.

The engineer views the corresponding types, data mappings, executable dependency graph and tests. The operations owner views required actions, responsible teams, timing and evidence of completion. A diagram or decision table is generated from the same adopted rules. Maintaining a separate hand-edited flowchart would create another translation boundary and another version to reconcile.

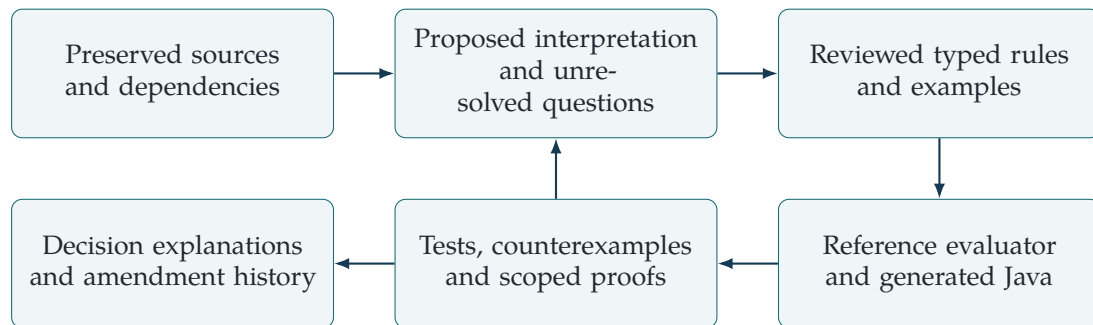


Figure 3: The proposed workbench. Counterexamples return to interpretation review. A proof result remains attached to the exact rules, assumptions and software version it checks.

5.2 The intermediate representation

A JSON document becomes an executable specification only when its expressions have defined meanings. RuleIR 0.1 fixes a finite, acyclic expression language and a separate event profile. The following sections give its expression, evidence, event, storage and service rules in the manuscript. The JSON schemas and docs/specs/v0.1/ files provide machine-checkable structures and detailed operational copies of those contracts. They do not select an alternative logic or leave missing-value behavior to the implementer.

Each rule needs more than its condition. It must identify the source document and revision, paragraph or page, exact supporting span, language, source digest and retrieval date. It must record the legal actor, activity, product, client class, relevant time and interpretation status. It must distinguish a derived fact from a duty, permission or prohibition. A duty also needs an actor, trigger, action, timing rule and evidence by which performance can be assessed. A missing source deadline remains missing; operational escalation times belong to a separately identified bank policy.

Definitions should be reusable and versioned. A field for portfolio value must refer to the adopted portfolio definition, relevant ownership rules, valuation date and currency conversion convention. The application mapping records where each fact comes from, its freshness and who can resolve a conflict. A rule whose necessary definition remains unresolved can be reviewed and discussed, but it cannot be released as an executable control for that use.

The complete financial-condition bundle is supplied in fixtures/spi-financial.bundle.json beneath the specification directory. It declares two money inputs, an unconditional scope for this isolated subcondition, the expression, interpretation record and a verified span in Annex 1 paragraph 3.1. The excerpt below is a complete expression node from that bundle. Node identifiers make its trace reproducible.

Listing 4: The portfolio comparison in the supplied JSON bundle. Amounts are HK cents.

```
{
  "node_id": "portfolio.ge", "op": "compare", "cmp": "ge",
  "left": {
    "node_id": "portfolio.input", "op": "fact",
    "name": "portfolio",
  },
  "right": {
    "node_id": "portfolio.threshold", "op": "literal",
    "type": "money_hkd", "value": "4000000000"}
}
```

The second comparison uses `net_assets_ex_home` and `"8000000000"`; an any node combines them. These inputs denote amounts normalized under adopted wealth definitions, not any account balance supplied under the same name. Release requires those definitions and applicability to be reviewed. The narrow result name, `spi.financial`, identifies a subcondition rather than a complete SPI assessment or trade authorization.

5.2.1 Types and the expressions an agent may generate

The scalar types are Boolean, integer, HKD money and Gregorian date. Integer and money values use canonical decimal strings; monetary units are cents. There are no floating-point values, implicit casts or known nulls. A date must be valid, including its leap-year constraints. Foreign-currency normalization records the rate, valuation instant and rounding policy before producing an HKD input. A missing exchange rate therefore leaves an unknown amount.

The expression grammar contains typed literals, fact and rule references, `all`, `any`, `not`, equality/inclusive/strict comparison, addition, subtraction, rational scaling, conditional choice and explicit defaults. Addition and subtraction require matching integer or money types. Scaling uses a nonnegative integer numerator and a positive denominator; a known result must divide exactly. A half-cent allocation returns `E_INEXACT_SCALE` until an external reviewed rounding policy resolves it. Date arithmetic is kept in the separate scheduler so ordinary library behavior cannot choose a legal anniversary convention unnoticed.

An implementer can express the grammar below directly as tagged records. Every `Expr` additionally carries a unique `node_id`. A rule is (`id`, `type`, `scope`, `body`, `interpretation_id`, `source_span_ids`); `scope` is Boolean and `body` has the declared result type.

Listing 5: Complete operator grammar for RuleIR 0.1; repeated arguments are ordered. The Java demonstration implements a stated subset.

```
Type = bool | integer | money_hkd | date
Expr = literal(Type, value)
      | fact(name) | rule(name)
      | all([Expr, ...]) | any([Expr, ...]) | not(Expr)
      | compare(eq | ge | gt, left, right)
      | add(left, right) | sub(left, right)
      | scale(arg, numerator, denominator)
      | if(condition, then_value, else_value)
      | default(base, [Exception, ...])
Exception = (exception_id, guard, value,
             interpretation_id, source_span_ids)
```

A comparison requires two matching integer, money or date operands. Dates compare in calendar order; Boolean comparisons are not admitted in this version. A conditional requires a Boolean condition and matching branch types. Default values match their base type, and all guards are Boolean. An empty exception list simply selects the base. There are no loops, quantifiers, dynamic names or arbitrary function calls. More elaborate assessment and aggregation

enter through separately reviewed fact mappings.

Every node has a unique identifier. References must resolve, types must agree and the complete dependency graph must be acyclic. A rule reference may refer only to an unconditional definition; a scoped result cannot quietly become a Boolean false in another expression. Authors can expose the scope explicitly as a named Boolean definition. Unknown operators and additional JSON properties are rejected. This keeps a model from extending the language merely by emitting a plausible field name.

5.2.2 Evidence is an input to computation, with its own history

A normalized fact is known, unknown or conflicting. Known facts have an exact value, evidence identifiers, a validity interval and a recorded timestamp. Unknown facts explain whether evidence is missing, stale or awaiting assessment. Conflicts preserve the disagreeing records. Two admissible records with the same value can combine their evidence; different values require resolution. Arrival order is not a ranking of legal authority.

Every request supplies `valid_at`, the instant being assessed, and `known_at`, the latest evidence the assessment may use. Intervals are half-open: an amount valid until midnight is unavailable at that midnight. A correction names the record it supersedes and becomes visible only when its recorded time is eligible. Thus a corrected assessment can use new information while the original decision remains reproducible.

Before Boolean evaluation, the engine checks conflicts in the static transitive dependency closure, including branches that might later be skipped. A conflicting portfolio amount therefore returns `CONFLICT` even if the net-asset route could establish the financial condition. An unrelated conflict does not block that rule. This conservative policy is a proposed engineering choice; it is deliberately different from ordinary unknown-value propagation.

5.2.3 Defaults, branch selection and failure behavior

A default contains a base value and named exceptions. Each exception has a Boolean guard, a value, an interpretation identifier and source spans. Evaluate all guards first. Two true guards produce a conflict, even if the proposed values agree. Otherwise any unknown guard yields an unknown result, because it might become a competing exception. With exactly one true guard and all others false, evaluate that exception's value. With all false, evaluate the base. Priority is expressed only through explicitly reviewed nesting, never row order.

Boolean combinations evaluate all operands in source order. A runtime error or exception conflict cannot disappear because a different operand is sufficient. A conditional evaluates its condition and then only the selected branch; an unknown condition yields unknown, even when both branch values happen to be equal. Skipped branches are still type checked. These choices eliminate implementation-dependent short-circuit behavior while retaining precise traces.

The rule's scope is evaluated separately. A false scope yields `OUT_OF_SCOPE`; an unknown scope yields `UNKNOWN`. In scope, Boolean bodies return `TRUE` or `FALSE`, other known bodies return `VALUE`, and missing evidence returns `UNKNOWN`. Conflicts and errors are distinct tagged results. An invalid version time is an error, not a negative assessment. No exception handler may fall back to false.

The response carries all encountered missing inputs and the conservative subset blocking the answer. If a known net-asset route establishes the alternative, the unknown portfolio remains visible but is not a blocker. For an unknown result, the union of encountered unknown de-

dependencies is reported; the prototype does not claim it is a mathematically minimal explanation. Traces record evaluated nodes in postorder, evidence, source spans and explicitly skipped branch roots. Explanations and diagrams are rendered from that trace.

5.3 A precise example: money, alternatives and unknown values

Let P denote the client’s portfolio amount under the adopted definition, and let N denote net assets excluding the primary residence. Both are valued at the relevant date and expressed in HK dollars under an explicit conversion policy. For complete, valid inputs, the financial condition is

$$F(P, N) = (P \geq 40,000,000) \vee (N \geq 80,000,000). \quad (2)$$

The source of the constants, inclusive comparisons and alternative is Annex 1, paragraph 3.1. Equation (2) formalizes that subcondition; its inputs incorporate the separate definitions in paragraphs 3.2–3.4 (SFC and HKMA, 2023b). RuleIR stores monetary amounts as integer HK cents encoded as decimal strings. Conversions must take place before the comparison, with the exchange-rate source and valuation instant recorded. Floating-point rounding must not silently move a boundary.

Bank evidence is often incomplete. Define the comparison with a missing amount to return U, and let T, F and U mean established, refuted and unresolved for this condition. For the alternative in Equation (2), use the following truth table:

$\vee$	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

(3)

This table follows the intended treatment of incomplete evidence: a known sufficient route establishes the alternative, both refuted routes refute it, and otherwise missing information can matter. With unknown P and $N = 90,000,000$, the second comparison is true, so $U \vee T = T$. With unknown P and $N = 70,000,000$, the result is $U \vee F = U$. Figure 4 gives a flowchart view.

The other Boolean operations need equally explicit tables. Under the proposed conservative three-valued semantics, conjunction is false if an operand is false, true if both are true, and otherwise unresolved; negation swaps true and false and preserves unknown. This semantics does not discover every conclusion implied by correlations among unknown facts. For example, a general expression $A \vee \neg A$ remains unresolved when A is unresolved under these tables. A more powerful completion-based analysis may establish additional conclusions, but it must be a specified refinement with its own evidence. No accidental change of logic should occur when switching backends.

Contradictory input records are handled before this evaluation. If two authoritative-looking records disagree on the same ownership allocation, the workbench creates an evidence conflict and identifies the disputed fact. It must not insert both as classical premises and infer arbitrary results from an inconsistent theory. Being outside a rule’s scope, having insufficient evidence, and detecting a data conflict are also distinct operational outcomes. These statuses should wrap the logical result rather than being encoded as false.

5.4 Obligations and event histories

The decision fragment cannot represent every continuing control. For consent, let s_i describe the recorded arrangement immediately before event e_i : its client, selected categories, agreed

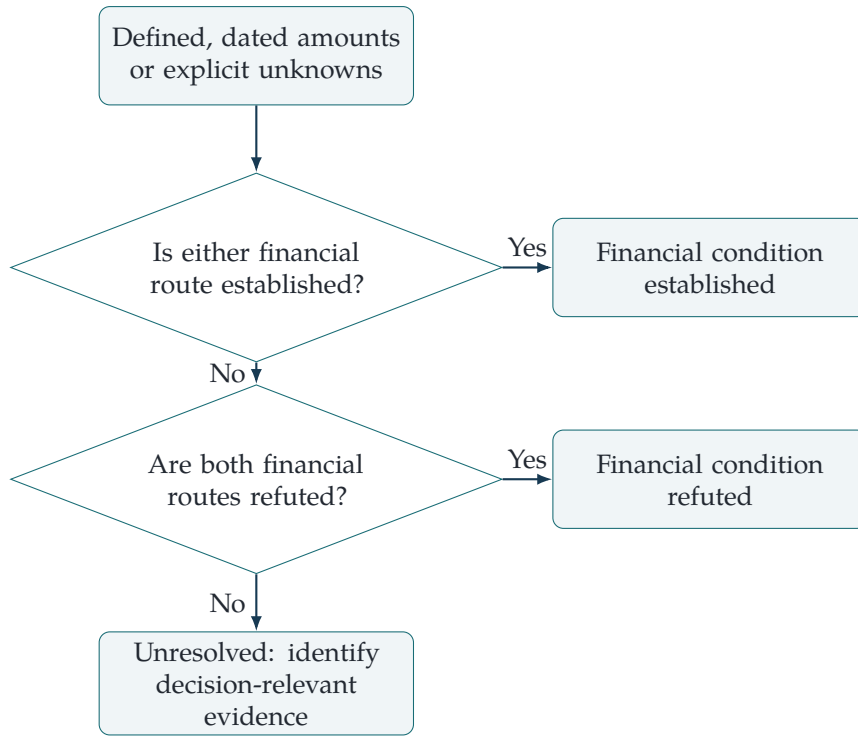


Figure 4: Proposed review flow for the financial subcondition, after resolving input conflicts. Passing this test alone does not establish SPI status.

threshold, monitoring mode, relevant reviews and withdrawal status. Each event includes its timestamps and ordering evidence. It changes the recorded state through a versioned transition function,

$$s_{i+1} = T_v(s_i, e_i). \quad (4)$$

Here i orders processed events; it does not assume equal physical time steps. Version v identifies the adopted rules and ordering convention. A withdrawal event can change the availability of the streamlined arrangement for subsequent decisions. A new-funds event can trigger the designated-account checks. An annual-review event creates work that must later be evidenced as completed. The exact consequences require review of the cited guidance and the bank's operating procedure (SFC and HKMA, 2023b,c).

An obligation record should distinguish its creation, due condition, performance, breach and any later remediation. If a transaction occurs after a withdrawal, the audit view must retain the transaction and identify the issue; it cannot erase the event because the authorization view would have blocked it. This follows the useful separation of normative permission from actual occurrence in eFLINT (van Binsbergen et al., 2026).

Time requires two histories. Legal validity time records when a source or adopted rule applies. System-recorded time records when the bank received and deployed it. These support different questions: what should have applied to an event, and what version did the system actually use? The same distinction applies to facts received late or corrected later. A reconstructed historical assessment may use new evidence, but it must not overwrite the original decision and its inputs.

Event ordering is a material assumption. The prototype should store source timestamps, receipt timestamps, sequence identifiers, time zone and any uncertainty in ordering. A synthetic trace with a consent withdrawal and execution at the same timestamp is unresolved unless the available ordering or an adopted convention settles it. Calendar computations should identify whether they use anniversaries, elapsed days or another reviewed operation, drawing on the

date semantics literature (Monat et al., 2024). Neither the operating system clock nor the order in which messages happen to arrive establishes legal priority.

5.4.1 The finite event profile

The first event implementation supports consent and non-preemptive achievement duties. Events have immutable identifiers, stream identifiers, occurrence and recorded timestamps, and an authoritative sequence number. Timestamps are UTC with microsecond precision; legal date conversion explicitly uses Asia/Hong_Kong. Equal occurrence times can use sequence order only when the connector declares that ordering authority. Otherwise the result is `E_ORDER_UNRESOLVED`. A duplicate event with identical bytes is a no-op; reuse of its identifier for different content is an error.

Consent begins as not established only when the stream is complete from its declared inception, or is complete after a reviewed initial snapshot. An incomplete export instead yields an unknown history. A grant creates an active consent generation; withdrawal makes it withdrawn; a new grant needs new evidence and creates a new generation. Observed transactions remain in the audit history even when the current consent state would prevent streamlined use.

An obligation records actor, action, subject, source version, activation and a deadline or explicit absence of a deadline. Let a be activation and d the exclusive deadline. Under this proposed profile, a matching performance at time p is timely when

$$a \leq p < d. \quad (5)$$

This is the chosen boundary convention for the profile, not an assertion that every SFC deadline has the same wording. A local due date is converted to the next local midnight. An anniversary requires an explicit leap-day policy; a reviewed choice of February 28, March 1 or rejection replaces an implicit library default. Business-day calendars are deferred.

Absence of a timely record establishes breach only when a watermark shows that the observation window through d is complete. The wall clock cannot create that evidence. Without completeness the obligation stays open. Timely performance produces `SATISFIED_ON_TIME`; a complete window without it produces `BREACHED`. A subsequent matching performance produces `PERFORMED_LATE` and retains the breach. A deadline-free request can be performed but does not become overdue through an invented number of days.

For example, consider a synthetic review activated at midnight UTC on September 1 and due at midnight on September 2. A record one microsecond before the latter instant is timely; a record exactly at it is late. The fixtures contain both. These dates illustrate the runtime convention rather than an SFC-prescribed review deadline. If a late-arriving record later proves timely performance, replay produces a corrected assessment with a new knowledge cutoff; the original assessment remains visible. Maintenance duties, pre-emptive satisfaction, waivers and legal compensation require future profiles and are rejected now.

5.5 What can be proved, and what must be reviewed

There are three distinct assurance questions. The first is interpretive: does the adopted specification represent the relevant sources for the chosen business scope? The second is computational: does the implementation compute that specification? The third is evidential: do the facts and event records represent the relevant circumstances? A complete change package needs evidence for all three, but the evidence is different.

To make the computational question precise, let R_v be an approved version of the formal rules. Let $\llbracket R_v \rrbracket(x)$ denote the reference meaning of those rules on input x , and let $C_v(x)$ be the result of an implementation or compiled adapter. Define D_v to include the declared input types, admissible values, explicit unknowns and relevant state. The target is

$$\forall x \in D_v : C_v(x) = \llbracket R_v \rrbracket(x). \quad (6)$$

Equality here includes the named decision and required obligations represented by the specification. If explanations are part of the verified output, their relationship must be stated too. The meaning of the output cannot shrink from a set of duties to one Boolean while retaining the same assurance claim.

Equation (6) is a proposed verification objective, not a result already established by LegalMath. For a finite input space it may be checked by enumeration; for a restricted expression language it may be amenable to an SMT encoding or a proof by induction over expressions. For a production adapter, the mapping from bank data into x and the implementation of calendar and currency operations also belong to the argument. A test suite samples the equation and provides useful defect evidence, but it does not generally prove its universal quantifier.

The interpretive question does not become a theorem simply by naming a formal target. Source text, legal context and justified interpretations determine R_v . Compliance review, source coverage, competing readings and independently constructed cases address that step. Likewise, a theorem cannot establish that a recorded balance or signature is authentic. These limits are operationally useful: each review task is directed to the person and evidence that can answer it.

5.6 Counterexamples make disagreements reviewable

For many regulatory changes, a small counterexample is more useful than a large proof transcript. Suppose the proposed implementation accidentally replaces the first inclusive comparison in Equation (2) with a strict one. Let that candidate be G , and write D for the declared set of admissible, complete inputs (P, N) . A solver can search for

$$\exists (P, N) \in D : F(P, N) \neq G(P, N). \quad (7)$$

The concrete values $P = 40,000,000$ and $N = 0$ witness the disagreement: F is true and G is false. A compliance officer can understand the error without reading the encoding. Replacing the alternative with a conjunction is similarly exposed by $P = 35,000,000$ and $N = 90,000,000$.

This technique also compares two candidate interpretations or old and new rule versions. It must check the admissibility and consistency of its assumptions first. If a test domain accidentally demands that the same amount is both positive and negative, the absence of a counterexample proves nothing useful about real cases. Unsatisfiable assumptions should be reported as a separate failure, with the conflicting constraints identified. Solver timeout and unknown results should never be rendered as “equivalent.”

The initial Z3 adapter supports a smaller language than the interpreter: Boolean, integer and money expressions with explicit knownness and bounded declared domains. Dates, scaling, defaults and events return UNSUPPORTED. Encode a Boolean expression a by two bits (t_a, f_a) : true is $(1, 0)$, false $(0, 1)$ and unknown $(0, 0)$; prohibit $(1, 1)$. Conjunction then has the encoding

$$t_{a \wedge b} = t_a \wedge t_b, \quad f_{a \wedge b} = f_a \vee f_b. \quad (8)$$

Disjunction uses the dual equations and negation swaps the bits. A numeric value carries a knownness bit, and a comparison produces neither Boolean bit unless both operands are known.

These equations implement the stated truth tables. Giving an unknown fact an unconstrained classical Boolean would instead make $A \vee \neg A$ always true and verify a different language.

The service checks domain satisfiability before searching for a difference in tagged results. Display a satisfying model only after replay through both interpreters. An unsatisfiable difference query yields `NO_COUNTEREXAMPLE_IN_DECLARED_DOMAIN`; an empty domain yields `INCONSISTENT_DOMAIN`; timeout stays `UNKNOWN`. Preserve formula, domain, rule hashes and solver version. A solver answer is a different class of evidence from a certificate replayed by an independent checker.

For event rules, the corresponding search concerns traces. A useful first property is that, in the adopted consent model, no transition representing a new streamlined decision uses a withdrawn consent state. Another is that processing the same evidence event twice does not create duplicate obligations. Each property has assumptions about event identity and ordering. Bounded trace search can find violations, while a proof of an invariant can establish a stronger claim if the transition model and induction obligations are checked.

5.7 The smallest useful execution architecture

The authoring runtime is a Python 3.11 reference interpreter, with JSON Schema and semantic validation, SQLite storage, content-addressed source files and a local FastAPI service. A server-rendered interface presents source, rule and case together. The interpreter walks a validated syntax tree; it does not execute model-generated Python. Sets, quantifiers, general recursion, arbitrary code, currency conversion and unbounded temporal formulas are outside RuleIR 0.1. An external, evidenced normalizer can supply a reviewed count or HKD value.

These restrictions keep common questions tractable and make the trusted code small enough to inspect. They also provide a natural place to evaluate the literature: Catala for the decision fragment, DMN as a possible Java integration comparison, and a Stipula-inspired state model for a bounded workflow. The project should not integrate all candidate engines before demonstrating one complete use case.

A reference interpreter implements the written execution semantics. A schema validator checks structure; a semantic validator checks expression types, resolved dependencies, source links and explicit exception relationships; a scenario runner evaluates reviewed examples. A solver adapter then searches for conflicts, unreachable branches and semantic differences. Generated code should come from a deterministic compiler for the supported fragment. At transaction time, evaluation uses an approved rule version and a recorded fact snapshot; language-model generation belongs to authoring and review.

An implementation adapter is accepted only after its treatment of missing values, numeric boundaries, priority, dates and obligations agrees with the reference semantics on the required checks. DMN's hit-policy choices and Rego's undefined results make this comparison substantive (Object Management Group, 2024; Open Policy Agent contributors, 2026). A backend that cannot express a required distinction should report that limitation instead of approximating it silently.

5.7.1 Source identity, review and storage

The source store preserves original bytes and an exact text derivative. The raw document and derivative each have a SHA-256 digest. A source span records the derivative hash, page, half-open character offsets and a hash of the quoted text. Extraction changes create a new derivative; they do not silently move an approved span. A reviewer can open the original page beside

the extracted passage. Scanned or poorly extracted clauses require visual confirmation before release.

The source register distinguishes circulars, annexes, FAQs, codes, bank policy and interpretation. Dependency rows record references, definitions, amendments and replacements; unresolved targets remain explicit. Publication date does not automatically establish effectiveness. Applicability records identify legal entity, regulated role, activity, product, client class and jurisdiction. A product-provider requirement cannot enter the bank's distributor rules merely because its title mentions a product the bank sells.

The bundle contains definitions, expressions, spans and interpretations. Its canonical JSON hash identifies the entire immutable revision. Review decisions live outside those bytes and approve that exact hash. The lifecycle is draft, in review, approved, released and retired; rejection returns a revision to draft, while an edit creates a new draft hash. Release requires distinct compliance and engineering reviewers, resolved blocking questions and dependencies, a supported effective interval and passing evidence for that hash. The author cannot approve their own revision under the proposed prototype policy.

SQLite transactions enforce review revisions and idempotency. A stale review update is rejected rather than overwriting another review. Content-addressed blobs are written before their database references; a missing referenced blob is an integrity error. Audit rows record accepted transitions. Hashes detect accidental corruption but do not make a database tamper-proof against an administrator able to rewrite both records and hashes. Enterprise identity, signed retention and access controls belong to the later bank deployment.

5.7.2 The small financial fixture within the complete dry run

Section 3 follows the complete selected transaction profile. The smaller conformance fixture imports 23EC35 and Annex 1, verifies their hashes and opens paragraph 3.1 beside the financial rule. Its two wealth definitions are review issues until the relevant source and data mappings are accepted. An engineer loads the supplied draft bundle and the synthetic f01 case: portfolio HK\$40 million and qualifying net assets zero. The trace evaluates both inclusive comparisons and their alternative, returning TRUE for `spi.financial`. It also identifies the exact source revision, input snapshot and engine version.

The reviewer next opens f02, one cent below the portfolio threshold, with the other route still false. This returns FALSE. In f07, the portfolio is unknown and net assets are HK\$70 million, so the answer is UNKNOWN. In f06, changing only net assets to HK\$90 million establishes the financial condition despite the still-missing portfolio. These cases let a compliance reader inspect the interpretation before any system rule is exported. A proposed strict comparison is challenged with f01.

Once dependencies and meanings are reviewed, the evidence report is attached to the bundle hash, reviewers approve it and the release service exports the package. The production change package includes deterministic Java source and its tested JAR against the same decision contract. A host adapter supplies the bank's facts and enforces the released-version and concurrency requirements. A changed rule creates another hash and invalidates release approval. Historic decisions still point to the old hash and their original evidence.

5.7.3 Service boundaries an implementer can use

The API imports sources, creates and retrieves bundles, records reviews, releases approved hashes, evaluates snapshots, appends events, replays histories and compares versions. The

transport contract is summarized below; companion files preserve the same field names for implementation. The central reference call is

```
evaluate(bundle, snapshot, rule_id, valid_at, known_at,
        mode="draft")
-> EvaluationResult
```

Its response includes the tagged outcome, exact value if any, reason codes, missing and blocking inputs, trace, source/bundle/input hashes and engine version. Production calls require a released exact hash; draft calls visibly retain draft mode. An HTTP failure is different from a well-formed request whose assessment returns an error or conflict.

The local authoring service uses the following endpoints. Path parameters name immutable content identities; state-changing requests also carry a caller-scoped idempotency key. Repeating the same key and request returns the stored response; reusing it with different content returns HTTP 409. Optimistic revisions prevent two reviewers from overwriting each other's changes.

Table 5: Authoring-service contract. This service is separate from the Java transaction library.

Method and path	Required request and returned result
POST /v1/sources/import	Official URL or local blob, reference ID and media type; returns source revision and derivative hashes. Invalid content is 422; fetch failure is 502.
POST /v1/bundles	Complete typed bundle; returns immutable hash, draft status and revision 1. Schema or type failure is 422.
GET /v1/bundles/{hash}	Returns exact bundle, review state and issues; unknown hash is 404.
POST /v1/bundles/{hash}/review	Decision, reviewer identity/role, comment and expected revision; returns new state/revision. Stale update is 409; invalid transition is 422.
POST /v1/bundles/{hash}/release	Evidence-report hash, Java-release-manifest hash and expected revision; returns release record if all guards pass. Missing approval or mismatched code evidence is 422.
POST /v1/evaluations	Bundle hash, snapshot, rule ID, mode and both times; returns the complete evaluation. A nonreleased production bundle is 409.
GET /v1/evaluations/{hash}	Returns preserved request, result and source links; unknown hash is 404.
POST /v1/event-streams/{id}/events	Event, expected revision and ordering authority; returns stored event/revision. An ID payload mismatch or sequence collision is 409.
POST /v1/event-streams/{id}/replay	Both times and profile version; returns immutable consent/obligation state with evidence.
POST /v1/comparisons	Old/new bundle hashes, rule ID and declared domain; returns a persisted job ID and later a counterexample or a specifically bounded analysis result.
GET /v1/changes/{old}/{new}	Returns changed definitions, rules, sources, affected dependants and evidence that must be regenerated.

The persistent relationships follow the questions the reviewer asks. A source revision points to immutable raw and extracted blobs; an interpretation points to its spans and unresolved issues; a bundle contains the interpreted rules; reviews and release records point to that bundle hash. A snapshot identifies its subject and facts. An evaluation points to the bundle, snapshot and both times. An event belongs to a stream and has an immutable identifier and ordering

evidence; replay results are separate records. Jobs, idempotency records and audit events preserve asynchronous work and accepted state transitions. The supplied SQL represents these relationships with foreign keys and JSON payloads; a Java release manifest can be retained as an immutable blob referenced from the release evidence. No mutable row is allowed to change the contents identified by an old digest.

For content identity, sort JSON object keys by ASCII order, preserve array order and string code points, use compact UTF-8 and hash the bytes with SHA-256. Reject duplicate keys, floating-point values, nonfinite values and unpaired Unicode surrogates. JSON integer metadata is limited to the exactly portable range $[-(2^{53} - 1), 2^{53} - 1]$; domain integers and money use decimal strings. This is an explicitly specified local format, not an assertion of conformance to a different canonical-JSON standard. A result hash excludes itself and incidental timing/request identifiers. These rules let the Python authoring service and Java library identify the same inputs and semantic output.

Longer solver or drafting calls use persisted jobs with explicit failed, cancelled and timed-out states. They cannot alter approval records. Official documents and model output remain untrusted input: fetching restricts hosts and redirects, validates content and limits resources; parsing never executes a document's instructions. The initial service binds to localhost and uses public sources and synthetic facts. This makes the proposed behavior implementable before private data access or production integration is considered.

5.8 Language-model drafting with visible uncertainty

The drafting service should propose clause segments, roles, definitions, cross-references, candidate rules and review questions. Every proposal must cite the exact source passage and expose the definitions it imports. The service should use constrained structured output, followed by validation against the rule grammar. Retrieved text is evidence to interpret, not an instruction that can modify the workbench or execute arbitrary tools.

Independent candidate translations can help discover uncertainty. Their value comes from meaningful differences, not a vote count. When two candidates disagree, the solver should try to construct a case that distinguishes them. When both depend on a missing FAQ, the next action is to retrieve and inspect that source. When the language requires legal judgment, the result is an explicit assessment task. The model's confidence and the solver's ability to execute a formula must remain separate from approval of the interpretation, consistent with the lessons of ARc and the tax-reasoning work (An et al., 2025; Jurayj et al., 2026).

The reviewer should receive a short set of consequential questions. For example: does the selected evidence establish that an agreement is absent, or merely that it has not been retrieved? Does a new circular replace the entire earlier source or a specified part? Does a control apply to the distributor or the provider? Question selection should reduce ambiguity in the actual decision rather than produce generic compliance checklists.

5.9 Amendments and the release package

Every source revision should produce both a textual change view and an impact view. The former identifies changed passages, annexes and links; the latter follows dependencies to interpretations, definitions, rules, tests, data mappings and deployed controls. The 2023 and 2026 tokenisation circulars form a useful initial replacement case. A replacement marker is evidence about the document relationship, but application dates and treatment of existing arrangements still require the relevant source and review (SFC, 2023b, 2026b).

Where old and new executable rules cover the same domain, the workbench should find representative cases on which their outcomes differ. Where a new rule introduces an additional fact or actor, it should identify the new data dependency and the cases that cannot yet be evaluated. A textual difference alone cannot measure operational impact, and an unchanged threshold cannot establish that a definition or exception is unchanged.

The release package should contain the source snapshot, interpretation record, rule version, unresolved exclusions, approved examples, test and solver results, adapter version, data mappings and owner approvals. Decisions made with that package should record its identifier and input snapshot. Dependency analysis can identify tests and interpretation evidence that remain relevant after a change. Every changed bundle nevertheless needs fresh approval of its exact hash and a new evidence report; a result for an older hash cannot authorize its release. The historical record remains intact. This distinction permits reuse of work without obscuring what needs fresh review.

6 Delivering a control to the bank's Java system

Java is the required deployment output. The authoring workbench may use Python for source processing, review and reference evaluation; a transaction must be able to call the released Java library without running Python or contacting a language model. The initial integration assumption is a plain Java 17 JAR. This keeps the decision code independent of Spring, application servers and particular rule engines. A bank on another Java version needs a declared target and the same conformance checks before receiving a compatible build.

6.1 What has actually been built for this proposal

The delivered SPI-Demo1 example contains a source-anchored RuleIR bundle, synthetic fact snapshots, a separately written Python evaluator, a deterministic Java emitter, the generated class, two small Java support classes, and a separate host caller. Its implemented expression subset is deliberately explicit: Boolean, integer and HK-cent literals and facts; unconditional rule references; all, any, not; and exact numeric eq/ge/gt comparisons. The root has a Boolean scope. Dates, arithmetic expressions, general defaults, conditionals and the full obligation language remain part of the proposed RuleIR implementation; the demo compiler rejects them. The demonstration response is its own smaller API profile, not a claim to implement every field and trace node of the full RuleIR service.

The distinction matters for an implementing agent. The example demonstrates an actual compiler and integration path that can be extended. It does not silently turn the 35 general RuleIR contract fixtures into executed tests. Those fixtures remain the completion target for the full interpreter and backend. The separate SPI example has 32 executed decision scenarios and eleven consent histories, with its own recorded evidence.

6.2 The input and output boundary

The public API accepts an immutable Snapshot, a time being assessed and a knowledge cutoff. A Snapshot binds a subject identifier to the declared fact map. A Fact carries a declared scalar type and one of known, unknown or conflict. A known Boolean is a Java Boolean; a known monetary or count value is a BigInteger. Its evidence identifiers, validity interval and recording time are mandatory. Unknown has a reason and no value. Conflict identifies competing evidence.

The host supplies every declared field, using an explicit unknown where evidence is unavailable. Omitting a field or supplying the wrong type is a malformed request, not a legal conclusion. Validity intervals are half-open: evidence valid until 10:00 does not establish a fact at 10:00. Evidence recorded after the knowledge cutoff is unavailable to that replay. The API takes canonical UTC timestamps with six fractional digits; conversion from a Hong Kong business timestamp belongs in the host adapter and must preserve the intended instant.

Listing 6: The library API and one dated monetary input.

```
DecisionRuntime.Result evaluate(
    DecisionRuntime.Snapshot snapshot,
    String validAt,
    String knownAt);

// A known value is exact and supported by dated evidence.
Fact portfolio = Fact.known(
    "money_hkd", new BigInteger("40000000000"),
    List.of("valuation.lee", "ownership.lee"),
    "2026-09-01T00:00:00.000000Z",
    "2026-10-01T00:00:00.000000Z",
    "2026-09-01T00:00:00.000000Z");

Fact missingPortfolio = Fact.unknown("money_hkd", "MISSING");
```

The first amount is HK\$40 million expressed in cents. A bank adapter reading a decimal amount should specify the currency, use exact decimal arithmetic and reject an unsupported fraction of a cent after the adopted conversion. In Java, `amount.movePointRight(2).toBigIntegerExact()` can enforce exact cents after that policy has been applied. Calling `doubleValue()` first would discard the property the specification is trying to preserve. For a currency conversion, the exchange-rate source, date, direction and rounding rule are part of the evidence mapping.

The result contains a logical status and an operational disposition:

Status	Disposition	Host meaning
TRUE	Conditions met	Selected streamlining conditions established; continue the other controls.
FALSE	Standard process required	At least one required condition is refuted; do not apply this streamlined route.
UNKNOWN	Review required	Evidence can change the conclusion; show blocking inputs.
CONFLICT	Review required	Resolve inconsistent evidence; preserve both records.
OUT_OF_SCOPE	Outside profile	Select another implemented profile or the standard process.

The demo's JSON includes profile, authority, both timestamps, the bundle and snapshot hashes, a postorder rule trace, missing and blocking input lists, and the result hash. A rule-trace entry gives its identifier, status, source-anchor identifiers and blocking inputs. Source anchors resolve through the bundle included with the release. Known values and their evidence remain in the preserved snapshot. The full RuleIR implementation additionally provides expression-node traces, standardized diagnostics and engine identity as specified in Section 5.

A malformed request throws an explicit Java input exception in this small library. The production adapter must map that exception, a timeout or an unavailable release to a technical-failure result and a review/standard-process path. It must never catch an exception and substitute a

successful Boolean. The demonstration result always carries `DEMONSTRATION_ONLY`; production authority requires a different, approved release record and enforcement by the host deployment process.

6.3 How the compiler emits final Java rather than a suggestion

The emitter first validates the JSON schema, references, identifiers, types and acyclic rule graph. It then assigns an internal Java method name to each rule and recursively emits its expression. Identifiers are validated data, never pieces of executable text supplied by a language model. The generated monetary comparison is represented by actual Java calls such as:

Listing 7: The generated financial method's essential body. The complete class is built and tested in the supplied example.

```
return c.rule("spi.financial", sourceIds, () ->
    any(
        compare("ge", c.fact("portfolio"),
            Value.of(new BigInteger("4000000000"))),
        compare("ge", c.fact("net_assets_ex_home"),
            Value.of(new BigInteger("8000000000"))))
    ));
```

`compare` returns a known Boolean when both operands are known, or an unknown with its blocking input names. `any` implements the truth table in Equation (3). Java evaluates its arguments before calling the method, so both branches have the specified eager evaluation and evidence collection. Replacing this with Java's short-circuit `||` would change the trace and missing-input behavior even where the final Boolean agreed. The context memoizes referenced rules and records their results in a stable order. A precheck detects conflicting facts in this profile's dependency set before normal expression evaluation.

The production emitter should implement one mapping per supported operator, with an explicit error for every unsupported construct. It must not ask an LLM to write new Java on each regulatory change. The LLM can propose the typed specification; after review, a deterministic compiler performs the mechanical translation. This concentrates semantic review on a small language and makes the generation step reproducible.

For the money comparison, the preservation argument is short. The IR represents both amounts as mathematical integers of HK cents. The Java boundary constructs the same integers as `BigInteger` values. The emitted comparison returns true exactly when `compareTo` is nonnegative, which expresses the same inclusive ordering. The argument depends on the input mapping preserving those integers. It would fail if the adapter truncated decimals, mixed currencies or overflowed a narrower integer before constructing the value. For a complete compiler, analogous local arguments must be composed for every operator, unknown/error case, rule reference and trace requirement. The delivered tests check examples of preservation; they are not that universal proof.

6.4 Reproduce the build and see the host result

The repository retains a project-local Temurin JDK 17 for this check, verified against the official archive checksum. It does not change the machine's default Java installation. From the repository root, the single build command is:

```
python3 examples/java-dry-run/build_and_verify.py \
  --jdk .localresources/java-toolchain/jdk-17.0.20.1+1
```

The Python authoring check uses the already available JSON Schema package. The generated Java library itself has no third-party runtime dependencies. The build uses `javac --release 17 -Xlint:all -Werror`; creates the JAR; compiles the separate fixture and host callers against that JAR; compares complete reference and Java results; and runs the three mutations. The package has fixed ZIP timestamps to make repeated builds with the same toolchain reproducible. The verification record stores compiler identity, source/compiler/JAR digests, the executed command, results and limitations.

The delivered files below are operational copies of the design, not additional reading needed to understand why it works:

File under examples/java-dry-run/	Purpose
spec/spi-control.bundle.json	Source-linked typed interpretation for the chosen profile.
spec/decision-cases.json; spec/consent-cases.json reference.py; compile_java.py	Independent expected statuses and synthetic evidence/history inputs. Separate reference evaluation and deterministic Java generation.
generated/GeneratedSpi.java src/main/java/hk/legalmath/spi/	Final generated decision class. Evidence, result, three-valued arithmetic/logic support and consent replay.
build/spi-controls-demo.jar generated/BankHostExample.java	Compiled library callable by a Java host. Complete synthetic host caller using the public API.
build/verification.json; build/host-output.txt	Executed validation and host results.

The separate host prints these dispositions for the synthetic history:

```
BEFORE_WITHDRAWAL STREAMLINING_CONDITIONS_MET
AFTER_WITHDRAWAL STANDARD_PROCESS_REQUIRED
RESULT_AUTHORITY DEMONSTRATION_ONLY
```

This is the concrete answer to how final code reaches Java: the bank depends on the released library, constructs the declared evidence snapshot through its adapter, calls the generated class and handles its structured result. The compiler, public circulars and drafting tools are build-time/review-time components. They need not be deployed into the bank's order-processing process.

6.5 The bank adapter and the exposure race

The adapter has four responsibilities. It selects the approved rule release for the legal entity, business profile and relevant time; resolves identities, categories and evidence into the declared inputs; obtains a consistent consent and exposure snapshot; and records the result alongside the transaction's other controls. The generated library should have no credentials for editing these source systems. A host record should include transaction identifier, release and snapshot hashes, category and consent generation, data-version identifiers, decision timestamps and the final disposition.

Concurrency is a material part of correctness. Suppose existing gross exposure is HK\$8 million

and two orders each add HK\$1.5 million against a HK\$10 million limit. If each reads HK\$8 million, both independently pass at HK\$9.5 million, but together they produce HK\$11 million. A perfectly correct generated formula does not prevent that race. The host must include reserved exposure and commit the new reservation under a transaction or optimistic version check. If the exposure or consent version changed after evaluation, it reloads and evaluates again. A withdrawal must invalidate a stale consent version before a new order can consume the decision. Market-exposure policy, release timing and retry limits must be agreed with the bank's order and risk owners.

An implementable host sequence is:

1. Resolve an idempotency key from the order request. A retry with the same key and unchanged payload returns its recorded disposition; a changed payload under that key conflicts.
2. Read the approved release and the subject/category consent generation, holdings, leverage and outstanding reservations with their version identifiers. Build the dated snapshot and calculate exposure including this order.
3. Evaluate the generated Java library and the host's remaining controls. If this profile is unresolved or refuted, do not consume its streamlined route.
4. Atomically validate the read versions, reserve the permitted exposure and append the decision record. On version conflict, repeat from the read step with a bounded retry policy; otherwise route to review.
5. Retain the committed record for replay. A later correction creates another assessment linked to the old one; it does not overwrite what was known earlier.

The standalone JAR demonstrates the decision step. It does not simulate this bank transaction protocol or certify concurrency in an unspecified host system. Those integration tests are required before production use.

6.6 Release contents, extension tasks and acceptance

A production change package must contain the approved source and dependency register, provision dispositions, rule bundle, reviewer records, supported profile and effective interval, generated Java source, runtime library, binary JAR, dependency/toolchain manifest, data-mapping contract, tests and verification report. The report names the exact JAR tested. The bank's deployment process verifies these digests and authorization, performs its security and change checks, then switches the active release atomically. A signature can authenticate the release authority; a digest alone only identifies content.

The implementation backlog adds a mandatory Java package, W03J, immediately after the full reference evaluator. It owns the Java domain/API classes, one emitter per RuleIR operator, result serialization, exact numeric/date behavior, compiled conformance harness, JAR manifest and host adapter contract. Its pass condition is equivalence of the complete supported responses on the full conformance suite, source-derived scenarios and distinguishing mutations, plus reproducible generation and a separately compiled caller. A backend that accepts an operator but approximates its meaning fails the package.

W04 must bind release evidence to both the rule bundle and the resulting JAR. W05 must extend the demonstration's consent replay to the full event contract, including obligation instances and the adopted calendar policy. W06 must show the source, interpretation, test case and Java result together. W10 must include host concurrency, idempotency, stale release and replay tests in its

engineering checks, then separately measure review effort and material errors in the pilot. This sequencing produces a Java deliverable early while keeping legal adjudication and production authority explicit.

7 A prototype that can answer a product decision

The prototype should answer whether a shared executable specification reduces material mistakes and review effort for a defined class of regulatory changes. It should also identify the cases that still require judgment and the cost of maintaining the formalization. A demonstration that translates a clean paragraph and returns a correct number would establish too little; a project that attempts all SFC circulars at once would make the source and evaluation problems too large to diagnose.

The proposed scope is a twelve-week exercise with three connected examples. The SPI circular and annexes supply the main decision and workflow case. The two tokenisation circulars supply a version-replacement and actor-specific case. The fund-marketing circular supplies exceptions and judgment-dependent clauses. The thematic-review announcement supplies a negative classification example: the collector must recognize useful regulatory information without inventing an eligibility rule (SFC and HKMA, 2023a; SFC, 2023b, 2026b, 2023a; SFC and HKMA, 2025).

7.1 Scope, team and deliverables

The planning assumption is one full-time implementation engineer, a part-time technical lead, a compliance specialist available for roughly two days per week, and part-time product/operations and independent review support. A formal-methods specialist should review the semantics and solver adapter at defined points. These are resource assumptions to validate at initiation, not estimates derived from measured implementation velocity. Reduced compliance availability should reduce scope or extend the schedule rather than silently transfer legal decisions to engineers.

The corpus should begin with the cited documents and follow the dependencies needed for the selected rules. The first discovery phase will choose a tractable slice, provisionally around 30–50 atomic rules and 150–250 synthetic or appropriately approved historical scenarios. These ranges are planning hypotheses, not known counts of requirements in the circulars. The corpus must include thresholds, exceptions, missing evidence, actor distinctions, dates and at least one continuing obligation. Entire related clause families should be reserved for held-out evaluation to reduce leakage from almost identical examples.

The demonstrable deliverable is a local workbench with source comparison, typed rules, examples, explanations and an amendment view. Its export is a reviewable package with generated Java, a compiled JAR, a deterministic decision API and evidence records. The initial runtime need not call any bank service: public text and synthetic client facts can demonstrate the full path. Integration design should identify the future data feeds and control owners, while actual access follows the bank's environment and information-handling requirements.

7.2 A startable implementation sequence

The implementation sequence below separates the delivered Java demonstration from the complete product modules. Its operational entry point is `docs/implementation/START-HERE.md`,

with corresponding source paths and commands in `work-packages.md`. The manuscript supplies the meanings, interfaces and acceptance conditions; these files are the implementation copies. W00 establishes an isolated Python 3.11 environment and locks actual dependency versions. It does not modify the environments of the three adjacent MCP projects. The proposed stack is Python’s exact integer/date/hash/SQLite facilities, JSON Schema, Pydantic/FastAPI, Poppler or a ResearchAssistant extraction adapter, Jinja2 for the review pages, and optional Z3 for authoring. Java 17, a deterministic source emitter and a compiled conformance harness are required for delivery. Package versions are resolved and recorded during bootstrap; the proposal does not claim those dependencies have been integrated already.

The following command works on the delivered specification pack now:

```
python3 scripts/check_spec_pack.py
```

It validates schemas, fixture structure, selected static constraints, the source anchor and the SQLite definition. After W03 implements the evaluator, the same checker has an explicit runtime mode:

```
.venv/bin/python scripts/check_spec_pack.py \
--runtime legalmath.conformance:evaluate_case
```

The second command is a future completion check for the full RuleIR interpreter, not an executed result. The narrower Java command in Section 6 is already executable. This distinction makes it possible to hand the proposal to an agent without confusing the written test expectations with a functioning compliance engine.

Table 6: Implementation packages and their completion evidence.
Detailed files, functions, commands and repair conditions are in the accompanying backlog.

Package	Work and demonstrable result	Evidence needed to proceed
W00	Package, locked environment, canonical JSON, immutable blobs and database migration.	Same content hashes across processes; duplicate keys/floats rejected; interrupted writes leave no broken references.
W01	Import 23EC35 and annexes; preserve derivatives, spans and dependencies.	Reproduce the paragraph 3.1 anchor; changed bytes and broken attachments are detected.
W02	Typed RuleIR, references, graph validation and dated evidence normalization.	Reject malformed/type-invalid/cyclic rules; explain missing, stale and conflicting facts.
W03	Pure evaluator and deterministic trace. First financial-condition demonstration.	Execute 35 decision fixtures; catch boundary, OR, unknown and conflict mutations.
W03J	Complete Java emitter, runtime support, compiled conformance and host boundary.	Full supported results agree with the reference; mutations fail; source and JAR builds reproduce; a separate caller runs.
W04	Exact-hash reviews, release guards, Java package export and amendment comparison.	Self-approval, stale reviews and wrong-bundle or wrong-JAR evidence fail; historic versions replay.
W05	Consent, one duty profile, calendar policies and complete-window evidence.	Eight event fixtures plus ordering, duplicate, leap-date and late-correction tests.

Package	Work and demonstrable result	Evidence needed to proceed
W06	Local API and source/rule/case/amendment screens.	Contract tests and a reviewer walkthrough; unresolved evidence remains visible.
W07	Restricted SMT encoding and counterexample service.	Boundary counterexample replays; empty domain and unsupported constructs cannot pass as equivalence.
W08	Paginated corpus tracking, clause coverage and bounded model drafting.	Drafting cannot approve; omissions scored against an independent source inventory.
W09	Optional isolated ResearchAssistant, MathDevMCP and DynareMCP adapters.	Exact command/version/hash evidence; structural checks never mislabeled as proofs.
W10	Registered comparative pilot, error analysis, cost and review-time study.	Source parity, independent adjudication, uncertainty and a scoped investment decision.

The reference interpreter comes before language-model drafting because it supplies a stable target for evaluating the draft. That ordering also measures whether the shared specification is useful even without generative assistance. The event model is introduced only after the decision language is coherent, so difficulties with consent histories can be separated from errors in money comparisons or source extraction.

W00–W04, including W03J, receive a provisional four-week allocation; W05–W08 receive another four, with review, integration, rework and the pilot bringing the allowance to twelve weeks. The earlier three-week first-stage estimate did not include a required Java backend. This revised allocation remains a planning assumption, with scope reduction or a schedule extension if full operator conformance takes longer. Each package specifies files, function signatures, dependent packages and tests. Its acceptance condition concerns behavior, not elapsed time. For example, W03 supplies `ir/evaluate.py` and `ir/trace.py`; W04 supplies `review/lifecycle.py` and `review/amendment.py`; W07 supplies `analysis/z3_encode.py` and `analysis/compare.py`.

The deployment interface is the generated Java library, checked against the reference decision API. Catala, L4 and DMN are later adapter comparisons against the same fixed cases, not prerequisites that leave the implementer waiting for a framework choice. A Stipula–KeY experiment remains optional and bounded to one workflow property. If its abstractions hide the relevant event behavior, report that result and continue the useful reference implementation.

7.3 Reusing ResearchAssistant, MathDevMCP and DynareMCP

The adjacent projects supply reusable patterns and potential adapters. They are not interchangeable components of a legal reasoner. Their local code and documentation were inspected at the revisions recorded in the source manifest (LegalMath research notes, 2026b). The proposal makes no change to those repositories and does not assume that their current interfaces implement the proposed legal semantics.

ResearchAssistant is useful immediately for source intake, local storage, parsing, discovery and structured review records. It was used in this investigation to discover literature and parse retained PDFs. The available extraction route used `pdftotext` and reported low confidence requiring manual review. LegalMath needs a regulatory layer above it: circular identifiers, annex relationships, paragraph anchors, versions, replacement links and an applicability inventory. PDF extraction quality must be assessed on the actual source layout, especially footnotes and

tables.

MathDevMCP is useful after a property has been formalized. Its inspected Lean route binds checks to an exact target and assumptions, and its result handling distinguishes diagnostic evidence from proof. Its source-bound obligations and resumable evidence structures can support the question in Equation (6). The legal adapter must define the rule language's semantics, numeric and date operations, and the exact theorem requested. Structural agreement between a formula and a code fragment is not a substitute for semantic equivalence. The inspected project describes an internal research status and has its own licensing terms; reuse within a bank needs the appropriate software and ownership review.

DynareMCP supplies ideas for investigative planning, source graphs and meaningful differences between model versions. Its economic-model parser and numerical solvers are not direct engines for SFC requirements. The reusable pattern is a reviewable change supported by a dependency graph, with unresolved alternatives preserved. A legal adapter would compare rules, definitions, scope and dates rather than macroeconomic equations.

The proposed search use is equally specific. Historical DynareMCP investigation records use UCB-style exploration to choose work; the currently inspected MathDevMCP derivation controller explicitly describes a different bounded evidence-ordering method, not MCTS. It would be inaccurate to label both as the same proof-search engine (LegalMath research notes, 2026b).

For LegalMath, a search node could contain a candidate interpretation and its unresolved questions. Actions might retrieve a referenced FAQ, request a counterexample, check a definition, or prepare a question for a compliance owner. A search score may prioritize expected information gain or reduction in review work. It cannot determine which interpretation is legally correct. A branch with more permissive outcomes or more successful solver calls receives no special authority. If several readings remain viable, the product retains the disagreement.

Start with a bounded work queue ordered by materiality and dependencies. MCTS or UCB scheduling becomes a candidate only if the prototype reveals a substantial search-allocation bottleneck. Its evaluation would compare useful questions resolved per unit of review effort under the same budget, while correctness continues to depend on source review and formal evidence. Transaction execution uses approved rules, with no exploratory search in the decision path.

7.4 Evaluation must measure both meaning and implementation

The principal engineering question is whether the workbench produces a more reviewable and dependable implementation of the selected requirements than the current process. The baseline must therefore be a faithful version of that process, with the same documents, permitted reference material, business scope and definition of a completed change.

Use four conditions: the existing manual process; a source-grounded LLM summary/checklist; a hand-authored specification with the workbench; and the same workbench with model-assisted drafting. The summary baseline tests whether a simpler product achieves the benefit. The two formalized conditions isolate the incremental value of the model. Optional proof/search enhancements form a later arm under the same information and budget constraints. Preparation of formal rules, examples and reusable definitions must be counted, rather than treated as free information available only to the assisted condition. This addresses the information-budget distinction exposed by the tax-reasoning literature (Jurayj et al., 2026).

Independent compliance reviewers should derive expected outcomes and required actions from the selected sources before seeing the candidate implementation. Where interpretation is contested, the expected outcome can be a review request or an explicitly adjudicated reading. Am-

biguity must not be removed merely to obtain a convenient deterministic label. Cases generated from the formal model serve a different role: they exercise its paths and mutations, and should be reported separately from the independent source-derived cases.

Material errors include applying a rule to the wrong actor or category, allowing a scoped action after a relevant withdrawal, losing a required obligation, inventing a regulatory restriction, and converting decision-relevant missing information into an established fact. The exact taxonomy and severity must be agreed in weeks 1–2. A correct financial subcondition must not be scored as an incorrect trade permission merely because the test harness confuses those outputs.

Table 7: Proposed evidence contract for the prototype evaluation.
These are prospective criteria, not results.

Evidence	Definition and use	Decision role
Material interpretation and decision defects	Independently adjudicated errors in source coverage, rule meaning, resulting decisions or obligations. No unresolved critical defect may be carried into a proposed operational scope.	Primary quality criterion; blocks promotion of the affected scope.
Review effort	End-to-end person-time for a completed, accepted change, including source work, corrections, escalations and reusable-rule preparation.	Primary business criterion, conditional on quality.
Unknown and review outcomes	Frequency, reason, downstream handling and time to resolution; distinguish necessary abstention from unhelpful failure.	Explanatory diagnostic and business cost; incorrect silent resolution is a quality defect.
Source and dependency completeness	Every in-scope rule has inspected support and resolved imports; every required selected clause is accounted for.	Release veto for the selected corpus. Does not establish completeness of all HK regulation.
Adapter agreement and mutations	Supported semantics agree; targeted changes in operators, scope, dates and exceptions are detected by the required checks.	Engineering veto on the adapter; mutation scores alone do not establish legal fidelity.
Reproducibility	A saved rule version, fact snapshot and event sequence reproduce the original result and explanation references.	Engineering veto on historical replay.
Latency and token/runtime cost	Measure on the specified local workload, including failed drafts and review retries.	Explanatory and operating-budget evidence; no quality trade-off is implied.

W10 proposes at least 30 independently adjudicated change tasks, provisionally 10 for development and 20 held out. This is a feasibility budget assumption, not a statistical power calculation. Split by circular family and amendment chain; if the five initial documents cannot supply a credible separation, expand the corpus before making a generalization claim. Two compliance readers derive the expected scope, duties and case outcomes; contested interpretations remain a separate stratum rather than disappearing from the denominator.

Report paired error and reviewer-time differences, with intervals that account for reviewer and document effects. Repeated calls on one paragraph are not independent regulatory changes. Assign comparable tasks, rotate conditions where feasible and record prior exposure to avoid giving the assisted condition an unearned learning advantage.

Zero unresolved critical false approvals or missed duties is a proposed release screen on the ad-

judicated cases. Passing it does not estimate a zero population error rate. Criticality is defined with the bank before unblinding. If too few independent families remain for a defensible comparison, report descriptive results and the additional sample needed rather than rank viable approaches.

A small pilot may establish useful failure cases without supporting a precise speed ranking. Zero observed material errors in a finite synthetic set does not establish a zero error rate in the bank's workload. Similarly, a high mutation score says the selected perturbations were detected; it does not show that every missing legal requirement would be found. Report these limits directly and use the evidence to select the next scope of validation.

The evaluation should pause if its source inventory is wrong, its expected outcomes are not independently supportable, records are corrupted, or conditions receive materially different information without accounting for it. Those are problems with the evaluation. A failed drafting strategy or event adapter is a candidate failure: it should trigger a specified repair and repeat of the affected checks, rather than abandonment of the whole product direction.

7.5 Targeted tests and counterexamples

The initial suite should include exact threshold boundaries, small amounts below them, and cases in which only one financial route holds. It should distinguish unknown from zero, inconsistent from missing evidence, an absent agreement from an unretrieved agreement, and client attributes from representative attributes. These cases test meaning that common "typical client" examples fail to expose.

Category tests should change only the relevant category or qualification route. Ownership tests should change only the allocation agreement or associate status. Temporal tests should change only the withdrawal sequence, review date or fund movement. These paired cases help reviewers identify the controlling condition. For facts designated irrelevant to a narrow rule, changing only those facts should leave its result unchanged. Such metamorphic checks must name their domain: adding wealth might be monotone for Equation (2), but does not establish monotonicity of every suitability or product-distribution decision.

Mutation tests should deliberately replace inclusive with strict comparisons, alternatives with conjunctions, category-specific predicates with global ones, conditional requests with unconditional duties, and named exceptions with generic fallbacks. Event tests should include duplicated, delayed and out-of-order records. The test report must identify which defect each case would catch. This borrows the defect-oriented lesson of CUTEcat without claiming that automated case generation establishes source completeness (Goutagny et al., 2025).

7.6 Economic value and operating cost

The business case should use measured labor and maintenance effort. Let K be the initial cost of constructing the selected corpus, workbench and integration. Let M be the average manual cost of a comparable completed change, and let A be its assisted cost including human review, model use, operation and allocated maintenance. If those costs remain comparable across changes and $M > A$, the simple break-even count is

$$n_{\text{break-even}} = \frac{K}{M - A}. \quad (9)$$

This follows by comparing manual cost nM with assisted cost $K + nA$ and solving $nM = K + nA$. It is a planning relation, not an estimated return. If $M \leq A$, this labor-saving model alone supplies no break-even case. Material-error prevention, audit response and amendment speed may

have additional value, but they should be measured separately rather than assigned unsupported monetary benefits.

Reusable definitions can lower later costs, while new product classes and ambiguous amendments can raise them. The prototype should record both effects. Paper benchmark token costs and tax penalties are not substitutes for the bank’s staffing costs, risk definitions or change volume. A credible investment decision may be to continue only for a narrow class of rule changes where the measured benefit is clear.

7.7 Risks that shape the implementation

The most serious risk is a wrong but internally consistent interpretation. Its controls are independent source review, exact evidence links, explicit alternatives and cases designed from the circular. Formal verification strengthens the implementation step but cannot repair that source error automatically.

The second risk is incomplete scope. A well-modeled clause can depend on an uncollected definition or a different entity’s obligations. The selected corpus therefore needs an applicability and dependency inventory, with unresolved imports blocking release of affected rules. The discovery service should distinguish “not found” from “does not exist.”

The third risk is operational mismatch. A correct financial expression supplied with stale valuations or an event history with missing withdrawals can still produce a harmful result. The prototype should make data freshness, actor identity, event ordering and reconciliation visible. A later bank pilot needs named owners for those feeds and a documented response to uncertainty.

The fourth risk is creating a maintenance burden greater than the one removed. Keeping one typed specification with generated views reduces duplication; adopting many engines at once increases it. Pin tool versions, preserve proof and test dependencies, and add backends only when a specific integration or assurance question justifies them. Sources remain accessible if a research tool becomes unavailable.

Public sources and synthetic cases suffice for this prototype. If later work uses client information, deployment location, access control, retention and model access must follow the bank’s applicable requirements. This is an architectural constraint on that later integration, not a reason to delay the current public source study. The initial decision is whether the evidence justifies such a pilot, with a defined scope and accountable owners.

8 The first acceptance cases and the research record

The proposal can be made concrete before connecting to any bank system. The following cases specify the initial demonstration’s intended behavior. Monetary inputs refer to the adopted financial definitions; the table does not imply that those facts alone establish SPI status. These are source-based design cases for independent compliance review. Execution evidence for the selected Java subset is reported separately below; this broader table is not claimed as completed.

Table 8: Initial acceptance scenarios for the selected scope.

Case	Synthetic fact or source change	Required demonstration
F1	Portfolio HK\$40m; qualifying net assets HK\$0.	Establish the financial condition via the inclusive portfolio comparison.

Case	Synthetic fact or source change	Required demonstration
F2	Portfolio HK\$35m; qualifying net assets HK\$90m.	Establish the financial condition via the alternative net-asset route.
F3	Portfolio unknown; qualifying net assets HK\$70m.	Leave the financial condition unresolved; identify missing portfolio evidence.
F4	Portfolio unknown; qualifying net assets HK\$80m.	Establish the financial condition via the known sufficient route.
F5	Assets HK\$120m, liabilities HK\$15m, primary residence HK\$30m.	Derive HK\$75m for the illustrated net-asset calculation; assess the separate portfolio route independently.
O1	HK\$60m joint portfolio with a non-associate; written client allocation 20%.	Use HK\$12m for this portfolio-share illustration; explain the agreement and actor roles.
K1	Transaction-experience route established only for one category; a different category proposed.	Do not propagate a global experience flag; inspect the alternative qualification routes and selected categories.
C1	A qualifying wealth record, but conservative investment objectives.	Explain the investment-objective exclusion; wealth alone does not establish SPI treatment.
E1	Designated-account monitoring selected; exposure rises above threshold without new funds.	Show the relevant FAQ conditions and restrictions; avoid a universal transaction-block rule.
E2	A withdrawal instruction precedes a proposed streamlined decision in the adopted event order.	Use the changed consent state and retain the original instruction and ordering evidence.
T1	A third-party verification clause with an SFC-request trigger.	Preserve the trigger; distinguish an absent request from missing request records.
T2	Old tokenisation source replaced by the 2026 circular.	Preserve both editions, record replacement, identify affected rules and reproduce historical decisions.
M1	Product-linked gift versus a discount of fees or charges.	Preserve the stated exception and the scope of the cited marketing provision.
M2	Daily dealing fund with weekly redemption versus a different daily cut-off.	Distinguish reduced dealing frequency from administrative procedure.
D1	A thematic-review announcement enters the source collection.	Classify and track the announcement without inventing a new numerical eligibility criterion.

Cases F1–F5 and O1 derive from Annex 1, paragraphs 3.1–3.4. K1 and C1 concern paragraphs 4–7. E1 requires paragraph 8.3 together with Annex 2, FAQ 6; E2 concerns the consent provisions and the explicitly adopted event-ordering model. T1–T2 refer to the cited tokenisation editions; M1–M2 to the marketing circular; D1 to the thematic-review announcement (SFC and HKMA, 2023b,c; SFC, 2023b, 2026b, 2023a; SFC and HKMA, 2025). This source mapping is part of the acceptance package and should be visible beside each case in the workbench.

8.1 Conformance cases and the research record

The proposal now has a machine-readable companion to the business scenarios. Under docs/specs/v0.1/fixtures/, 35 decision cases specify complete bundles, synthetic snapshots, assessment times and expected result projections. Six deliberately invalid variants exercise schema, reference, type and identifier failures. Eight event histories cover withdrawal, regrant, timely performance, deadline equality, missing observations and late performance. Four release cases

distinguish valid approval from self-review, open issues and stale evidence.

These sets serve different purposes. The 35 cases specify the selected language; the fifteen business scenarios above identify regulatory interpretation and operational questions, several of which remain beyond the first executable slice. An agent must not claim that passing the language cases completes those business scenarios. For example, a Boolean assessment input can represent a reviewer's judgment about investment objectives, but it does not automate that judgment. Likewise, ownership and category definitions require their own reviewed normalization and end-to-end tests before release.

The delivered checker validates all three JSON schemas, the 35 decision fixture contracts, six negative examples, event-record structure, original/derivative/span hashes and the SQLite table definitions. It does not execute the planned decision, event or release engines. Its report says so explicitly. W03, W04 and W05 provide the corresponding full-runtime acceptance commands. The separate Java demonstration now supplies narrower execution evidence: 32 SPI decisions and eleven consent histories agree with the independently written reference; all three compiled mutations are detected; a fresh rebuild produces an identical JAR; and a separately compiled host calls that JAR before and after withdrawal. The exact commands, toolchain and source/binary hashes are retained in its verification report. Its ten rules and reviewed-input boundaries are explained in Sections 3 and 6. Independent legal adjudication, human usability assessment and published-tool proof replay remain prospective.

8.1.1 A reproducible research collection

The papers are stored under docs/papers/ using the requested filename convention: the title, a comma, the first author's surname and the year in parentheses. Filesystem-unsafe title punctuation is normalized, while the manifest retains the original title. Each record includes its source URL, bibliographic identity, edition note, retrieval date, byte count, PDF page count and SHA-256 digest. The library index distinguishes distinct works from alternate editions and identifies discovery leads that were not available as usable full text.

The technical notes identify which part of each paper supports a proposed borrowing and what remains untested. They distinguish a semantics from its implementation, a reported experiment from a reproduced result, and a publicly available repository from a licensed and approved bank dependency. Full-text extraction is retained for local search, but the source PDF remains the reference where layout or equations matter. ResearchAssistant extraction is an aid to inspection, not certification of the extracted text.

The SFC source snapshots and the earlier local-repository inspection records remain under .localresources/, with their own manifest. The LaTeX and bibliography are in docs/proposal/. This arrangement permits another reviewer to trace the proposal's substantive examples back to a document edition without reconstructing the web searches. Redistributing the paper library is a separate matter from using the local copies for this research; the source licenses and access terms remain relevant.

8.2 Decisions needed at the start of a bank prototype

The first decision is the precise business perimeter: legal entity, activity, client class and product family. The second is the owner of each interpretation and the independent review process for disputed cases. The third is the mapping from regulatory definitions to actual bank records, including freshness and missing-evidence treatment. These decisions should be made on the selected source slice, with worked examples, rather than as abstract architecture questions.

The initial technical commitment should be modest and testable: one source repository, one typed decision semantics, one bounded event model, one review interface and one deployment adapter. The literature gives substantial guidance for each part. It also explains why a universal translation-and-proof promise would be unsound. A successful prototype would show which interpretations are adopted, which computations follow them, which facts support the result, and where human judgment is still required.

References

- Accord Project. Cicero, 2026. URL <https://github.com/accordproject/cicero>. Official repository documentation and stated licensing inspected September 2026.
- Chenyang An et al. A Neurosymbolic Approach to Natural Language Formalization and Verification. arXiv:2511.09008, 2025. URL <https://arxiv.org/pdf/2511.09008v2>. First posted 2025; reviewed v2, 13 July 2026.
- Grigoris Antoniou, David Billington, Guido Governatori, and Michael J. Maher. Representation Results for Defeasible Logic. arXiv:cs/0003082, 2000. URL <https://arxiv.org/pdf/cs/0003082>. Reviewed 2000 preprint; journal version published in 2001.
- Apache KIE contributors. Drools, 2026. URL <https://github.com/apache/incubator-kie-drools>. Official repository documentation inspected September 2026.
- Joaquín Arias, Mar Moreno-Rebato, José A. Rodríguez-García, and Sascha Ossowski. Automated legal reasoning with discretion to act using s(LAW). Artificial Intelligence and Law, 2024. URL <https://arxiv.org/pdf/2401.14511>. 2024 arXiv copy; journal first published online in 2023.
- Tara Athan, Guido Governatori, Monica Palmirani, Adrian Paschke, and Adam Wyner. Legal Interpretations in LegalRuleML. CEUR Workshop Proceedings, 1296, 2014. URL <https://ceur-ws.org/Vol-1296/paper2.pdf>. Workshop paper.
- Catala contributors. Catala: compiler and core formalization repository, 2026. URL <https://github.com/CatalaLang/catala>. README, formalization notes and selected implementation/proof source inspected September 2026; proof not replayed.
- Silvia Crafa. From Legal Contracts to Legal Calculi: the code-driven normativity. Electronic Proceedings in Theoretical Computer Science, 368, 2022. URL <https://arxiv.org/pdf/2209.02353>. Author preprint.
- Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. Pacta sunt servanda: legal contracts in Stipula. arXiv:2110.11069, 2021. URL <https://arxiv.org/pdf/2110.11069>. Reviewed three-author 2021 preprint; later journal version has a different author list.
- Giorgio Delzanno, Cosimo Laneve, Arnaud Sangnier, and Gianluigi Zavattaro. Clause-reachability is undecidable in legal contracts. International Journal on Software Tools for Technology Transfer, 28, 255–275, 2026. URL <https://link.springer.com/content/pdf/10.1007/s10009-026-00841-5.pdf>. Published 19 March 2026; June issue.
- Christopher A. Esterhuyse, Tim Müller, and L. Thomas van Binsbergen. eFLINT Clingo implementation and evaluation artifact, version 2, 2025a. URL <https://zenodo.org/records/15470286>. README, compiler and test conventions inspected; documented exclusions retained; not executed.
- Christopher A. Esterhuyse, Tim Müller, and L. Thomas van Binsbergen. A Stable Model Semantics for eFLINT Norm Specifications and Model Checking Scenarios. GPCE 2025, 80–93, 2025b. URL <https://pure.uva.nl/ws/files/270385564/3742876.3742882.pdf>. University repository copy with cover; distinct semantics from the earlier interpreter.

- Aditya Ghose and George Koliadis. Auditing Business Process Compliance. *Business Process Management (BPM 2007)*, 2007. URL https://link.springer.com/content/pdf/10.1007/978-3-540-74974-5_14.pdf. Publisher chapter.
- Pierre Goutagny, Aymeric Fromherz, and Raphaël Monat. CUTECat: Concolic Execution for Computational Law. *Programming Languages and Systems (ESOP 2025)*, 2025. URL https://link.springer.com/content/pdf/10.1007/978-3-031-91121-7_2.pdf. Publisher open-access chapter.
- Guido Governatori and Antonino Rotolo. Logic of Violations: A Gentzen System for Reasoning with Contrary-To-Duty Obligations. *Australasian Journal of Logic*, 4, 193–215, 2006. URL <https://ojs.victoria.ac.nz/ajl/article/download/1780/6684>. Title-page publication 2006; migrated index year 2018 is not used.
- Guido Governatori, Francesco Olivieri, Antonino Rotolo, and Simone Scannapieco. Computing Strong and Weak Permissions in Defeasible Logic. arXiv:1212.0079, 2012. URL <https://arxiv.org/pdf/1212.0079>. Reviewed December 2012 preprint; later journal DOI.
- Guido Governatori, Mustafa Hashmi, Ho-Pun Lam, Serena Villata, and Monica Palmirani. Semantic Business Process Regulatory Compliance Checking Using LegalRuleML. *EKAW 2016*, 746–761, 2016. URL <https://inria.hal.science/hal-01572441/document>. HAL author manuscript.
- Neel Guha et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. arXiv:2308.11462, 2023. URL <https://arxiv.org/pdf/2308.11462>. Reviewed v1 including detailed task and evaluation appendices; no benchmark rerun.
- Mustafa Hashmi, Guido Governatori, and Moe Thandar Wynn. Normative requirements for regulatory compliance: An abstract formal framework. *Information Systems Frontiers*, 18, 429–455, 2016. URL <https://eprints.qut.edu.au/83732/23/Main.pdf>. Author manuscript; online publication 2015, issue 2016.
- Nils Holzenberger and Benjamin Van Durme. Connecting Symbolic Statutory Reasoning with Legal Information Extraction. *Proceedings of the Natural Legal Language Processing Workshop*, 2023. URL <https://aclanthology.org/2023.nllp-1.12.pdf>. ACL Anthology proceedings copy.
- Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. A Dataset for Statutory Reasoning in Tax Law Entailment and Question Answering. arXiv:2005.05257, 2020. URL <https://arxiv.org/pdf/2005.05257>. Dataset uses simplified statutes, not current operative US tax law.
- Liane Huttner and Denis Merigoux. Catala: Moving Towards the Future of Legal Expert Systems. *Artificial Intelligence and Law*, 2022. URL <https://inria.hal.science/hal-02936606/document>. Accepted manuscript, 10 August 2022.
- Reiner Hähnle, Cosimo Laneve, and Adele Veschetti. Formal Verification of Legal Contracts: A Translation-based Approach (Extended Version). arXiv:2509.20421, 2025. URL <https://arxiv.org/pdf/2509.20421v2>. Extended version, v2, 26 September 2025.
- William Jurayj, Nils Holzenberger, and Benjamin Van Durme. Language Models and Logic Programs for Trustworthy Tax Reasoning. arXiv:2508.21051, 2025. URL

<https://arxiv.org/pdf/2508.21051v3>. Alternate extended version of jurayj2026, v3, 5 February 2026; first posted 2025.

William Jurayj, Nils Holzenberger, and Benjamin Van Durme. Language Models and Logic Programs for Trustworthy Tax Reasoning. Proceedings of AAI, 40(45), 38688–38698, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/download/41212/45173>. Published proceedings copy.

Alan Khoja, Martin Kölbl, Stefan Leue, and Rüdiger Wilhelmi. Automated consistency analysis for legal contracts. Artificial Intelligence and Law, 2025. URL <https://link.springer.com/content/pdf/10.1007/s10506-025-09456-8.pdf>. Publisher version.

Yuta Koreeda and Christopher D. Manning. ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. Findings of EMNLP 2021, 2021. URL <https://aclanthology.org/2021.findings-emnlp.164.pdf>. ACL Anthology proceedings copy.

Robert Kowalski, Jacinto Dávila, Galileo Sator, and Miguel Calejo. Logical English for Law and Education. Prolog: The Next 50 Years, 2023. URL <https://www.doc.ic.ac.uk/~rak/papers/Logical%20English%20for%20Law%20and%20Education%20.pdf>. Author-hosted version; third author is Galileo Sator as on title page.

Cosimo Laneve. Liquidity analysis in resource-aware programming. Journal of Logical and Algebraic Methods in Programming, 135, 100889, 2023. URL https://www.cs.unibo.it/~laneve/papers/Stipula_LiquidityFULL.pdf. Reviewed author preprint dated 5 October 2022; journal edition 2023.

Cosimo Laneve. The Stipula Platform: A Workbench for Programming and Analyzing Legal Contracts. Formal Methods for Industrial Critical Systems, 2026. URL <https://www.cs.unibo.it/~laneve/papers/StipulaPlatform.pdf>. Author-hosted manuscript available September 2025; publisher chapter indexed 2026.

Cosimo Laneve, Alessandro Parenti, and Giovanni Sartor. Legal Contracts Amending with Stipula. Author-hosted manuscript, 2023. URL https://www.cs.unibo.it/~laneve/papers/H0_Stipula_llncs.pdf. 2023 author-hosted edition; do not identify this PDF with the different 2024 chapter Programming Contract Amending.

Legalese contributors. L4 compiler, IDE and DEONTIC reference, 2026. URL <https://github.com/legalese/l4-ide>. README and doc/reference/regulative/DEONTIC.md inspected; no execution or equivalence claim.

LegalMath research notes. Paper library, technical reading notes and retrieval manifest. Local files: docs/papers/README.md, reading-notes.md and manifest.json, 2026a. 38 PDF editions representing 37 works; expanded notes and coverage ledger; research snapshot 21 September 2026.

LegalMath research notes. Local project capability and source review. Local files: docs/sfc-rule-translator-design.md and .localresources/source-manifest.json, 2026b. MathDevMCP 655a9efeb338, DynareMCP 4bac9dd9733c, ResearchAssistant 9c0008f5c41b; inspected September 2026.

Lexpedite. Blawx, 2026. URL <https://github.com/Lexpedite/blawx>. README describes educational and experimental use; inspected September 2026.

- David Makinson and Leendert van der Torre. Input/Output Logics. *Journal of Philosophical Logic*, 29, 383–408, 2000. URL <https://icr.uni.lu/leonvandertorre/papers/jpl00.pdf>. Author-hosted manuscript.
- Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages*, 5 (ICFP), 2021. URL <https://arxiv.org/pdf/2103.03198v2>. arXiv v2, 2 July 2021.
- Raphaël Monat, Aymeric Fromherz, and Denis Merigoux. Formalizing Date Arithmetic and Statically Detecting Ambiguities for the Law. *Programming Languages and Systems (ESOP 2024)*, 2024. URL https://link.springer.com/content/pdf/10.1007/978-3-031-57267-8_16.pdf. Publisher open-access chapter.
- OASIS LegalRuleML Technical Committee. LegalRuleML Core Specification Version 1.0, 2021. URL <https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html>. OASIS Standard, 30 August 2021.
- Object Management Group. Decision Model and Notation, Version 1.5, 2024. URL <https://www.omg.org/spec/DMN/1.5/PDF>. Formal specification; hit-policy discussion in section 8.2.11.
- Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. *Proceedings of EMNLP 2023*, 2023. URL <https://aclanthology.org/2023.emnlp-main.313.pdf>. ACL Anthology proceedings copy.
- Open Policy Agent contributors. Policy Language: Rego, 2026. URL <https://www.openpolicyagent.org/docs/policy-language>. Official documentation inspected September 2026.
- OpenFisca contributors. OpenFisca Core, 2026. URL <https://github.com/openfisca/openfisca-core>. Official repository documentation inspected September 2026.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. *Findings of EMNLP 2023*, 2023. URL <https://aclanthology.org/2023.findings-emnlp.248.pdf>. ACL Anthology proceedings copy.
- M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. The British Nationality Act as a Logic Program. *Communications of the ACM*, 29(5), 370–386, 1986. URL <https://www.doc.ic.ac.uk/~rak/papers/British%20Nationality%20Act.pdf>. Author-hosted scan.
- SFC. Circular on distributors providing additional returns and other services or arrangements when marketing SFC-authorised funds. 24 October, reference 23EC46, 2023a. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC46>.
- SFC. Circular on tokenisation of SFC-authorised investment products. 2 November, reference 23EC53, 2023b. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC53>. Public entry marked as replaced by the circular of 20 April 2026.

- SFC. Circulars: Product Authorization, 2026a. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/products/product-authorization/>. Public index inspected 21 September 2026; not a complete private-bank regulatory inventory.
- SFC. Circular on tokenisation of SFC-authorized investment products. 20 April, reference 26EC22, 2026b. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=26EC22>.
- SFC and HKMA. Joint circular to intermediaries: Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors. 28 July, reference 23EC35, 2023a. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC35>.
- SFC and HKMA. Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors: Annex 1. Annex to joint circular 23EC35, 28 July, 2023b. URL <https://apps.sfc.hk/edistributionWeb/api/circular/openAppendix?lang=EN&refNo=23EC35&appendix=0>.
- SFC and HKMA. Frequently asked questions: Streamlined approach for sophisticated professional investors, Annex 2. Annex to joint circular 23EC35, 28 July, 2023c. URL <https://apps.sfc.hk/edistributionWeb/api/circular/openAppendix?lang=EN&refNo=23EC35&appendix=1>.
- SFC and HKMA. Joint circular to intermediaries: Concurrent SFC-HKMA thematic review of the distribution of non-exchange traded investment products. 29 September, reference 25EC48, 2025. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=25EC48>.
- Sepehr Sharifi, Alireza Parvizmosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. Symboleo: Towards a Specification Language for Legal Contracts. IEEE International Requirements Engineering Conference 2020, 2020. URL https://drive.google.com/uc?export=download&id=1WXwXeLrZdaJjhSJcCrt_wBXxDvhFkq2k. Author preprint linked by official Symboleo repository; some ancillary semantics references remain anonymized.
- Smart Contract Modelling, University of Ottawa. Symboleo editor and model checker, 2026. URL <https://github.com/Smart-Contract-Modelling-uOttawa/Symboleo-Model-Checker>. Official editor README, sale example and model-checker README inspected; no nuXmv replay.
- Stipula contributors. Stipula implementation and workbench, 2026a. URL <https://github.com/stipula-language/stipula-workbench>. Official project repositories; workbench and prototype documentation inspected September 2026.
- Stipula contributors. Stipula-to-Java/JML verification implementation, 2026b. URL <https://github.com/stipula-language/stipula-KeY>. Official translation repository; Translator.java and documentation inspected, no KeY replay.
- Silvano Colombo Tosatto, Guido Governatori, and Pierre Kelsen. Business Process Regulatory Compliance is Hard. IEEE Transactions on Services Computing, 8(6), 958–970, 2015. URL <https://eprints.qut.edu.au/100964/1/ComProof%202.pdf>. Author manuscript; DOI online year 2014, issue year 2015.
- L. Thomas van Binsbergen, Lu-Chi Liu, Robert van Doesburg, and Tom van Engers. eFLINT: a Domain-Specific Language for Executable Norm Specifications. GPCE 2020, 2020. URL <https://ir.cwi.nl/pub/29922/29922.pdf>. CWI author copy.

L. Thomas van Binsbergen, Christopher A. Esterhuyse, and Tim Müller. Reflections on the design, applications and implementations of the normative specification language eFLINT. *Journal of Computer Languages*, 2026. URL <https://arxiv.org/pdf/2511.12276v3>. Reviewed arXiv v3, 4 August 2026; journal title differs slightly; first preprint 2025.

Olivia Peiyu Wang, Sanna Wong-Toropainen, Daneshvar Amrollahi, Ryan Bai, Tashvi Bansal, Arush Garg, and Leilani H. Gilpin. Know Your Limits: On the Faithfulness of LLMs as Solvers and Autoformalizers in Legal Reasoning. arXiv:2606.16118, 2026. URL <https://arxiv.org/pdf/2606.16118v2>. v2, 19 June 2026; under review; methodological caveats retained.

Z3 contributors. Z3 theorem prover, 2026. URL <https://github.com/Z3Prover/z3>. Official repository documentation inspected September 2026.